

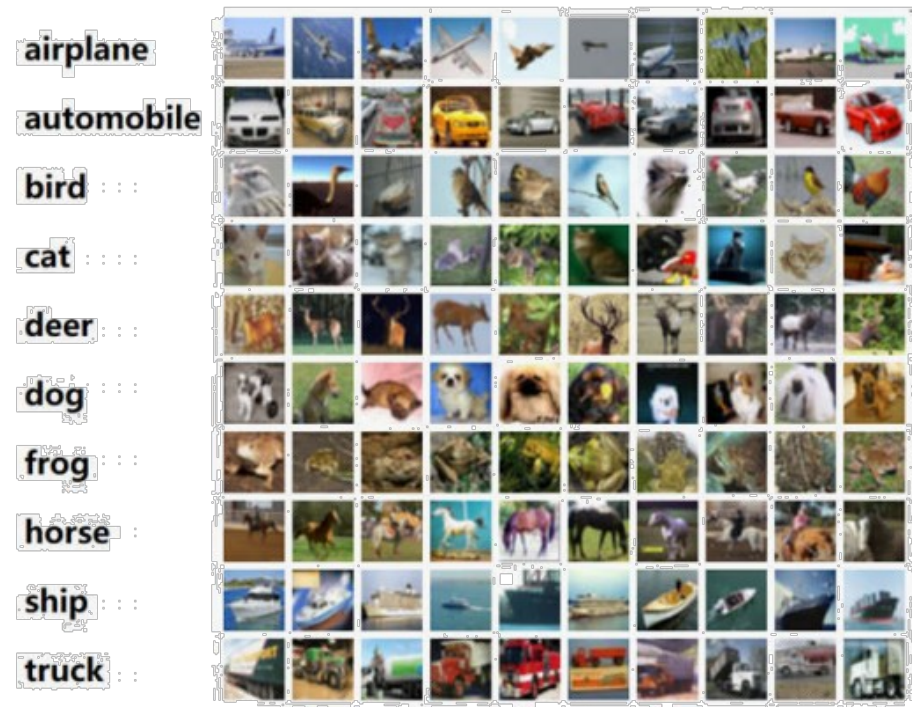
Linear classifier

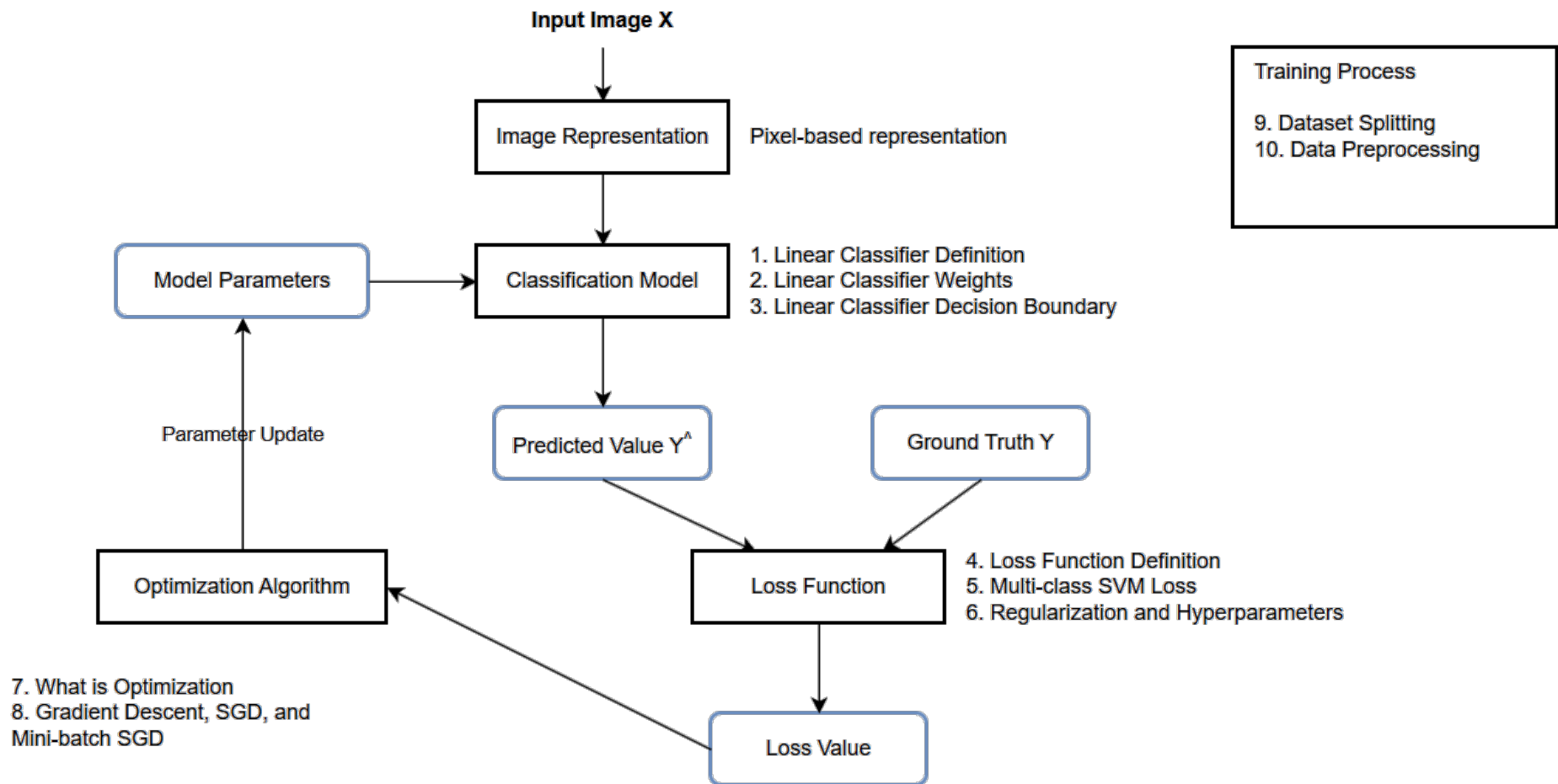
CHEN PENG
SCHOOL OF ARTIFICIAL INTELLIGENCE
AND DATA SCIENCE
HEBEI UNIVERSITY OF TECHNOLOGY



Dataset Introduction

- CIFAR10 dataset:
- Contains 50,000 training samples and 10,000 test samples ;
 - divided into ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.
 - The images are color images with a size of 32x32.





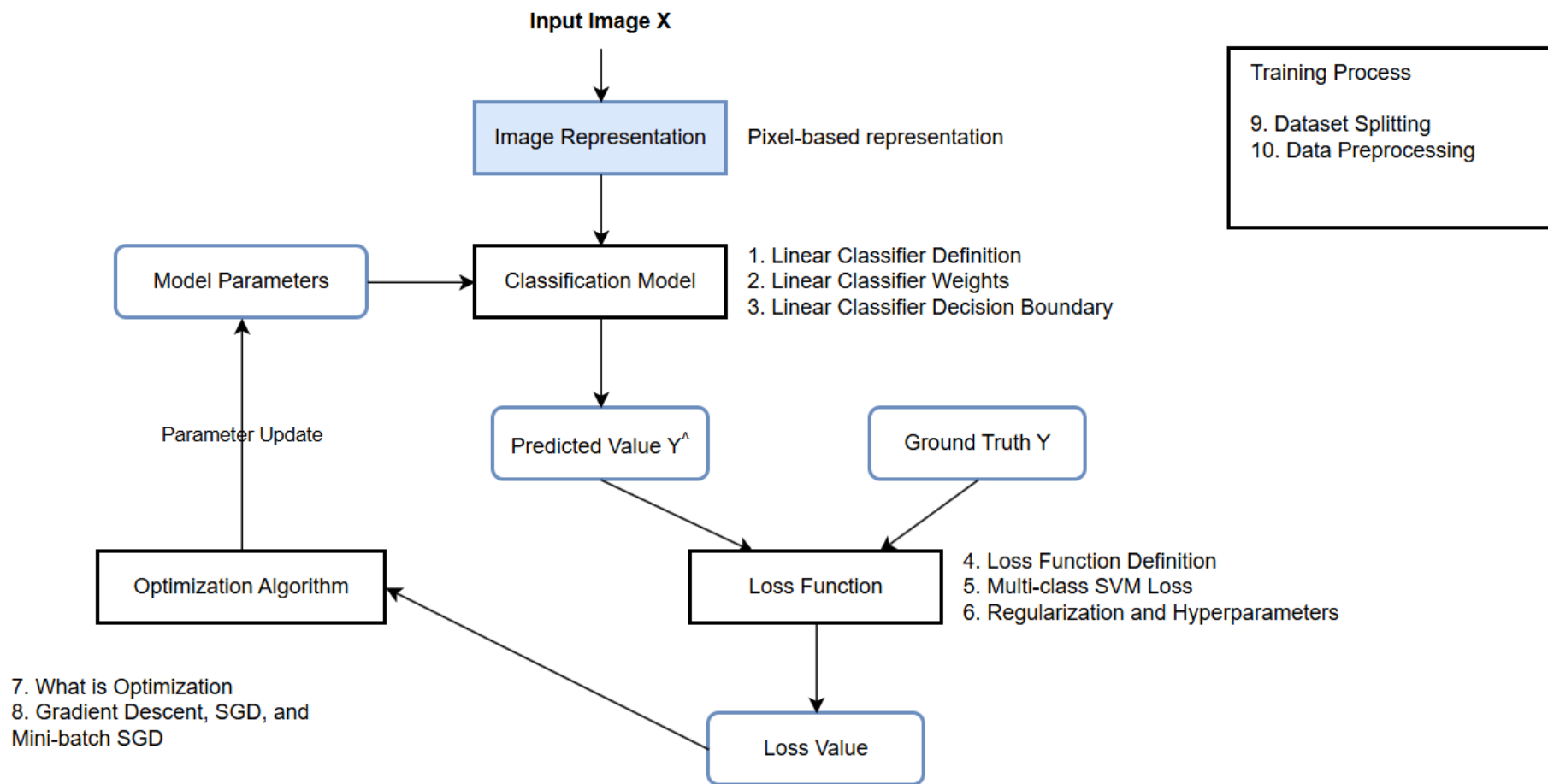


Image type

Binary



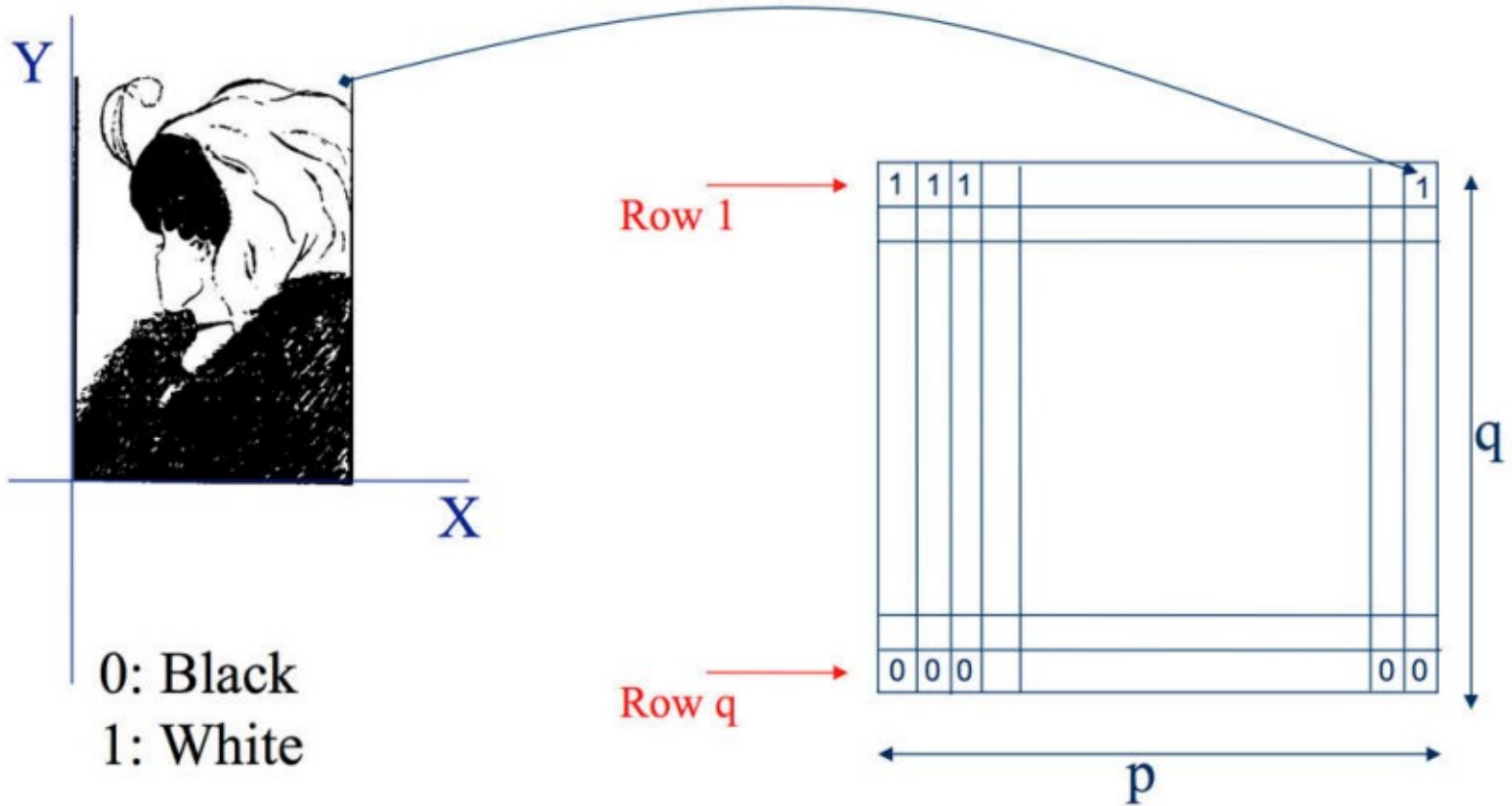
Gray Scale



Color



Binary image



Color image

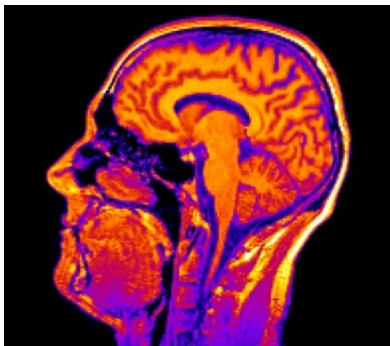
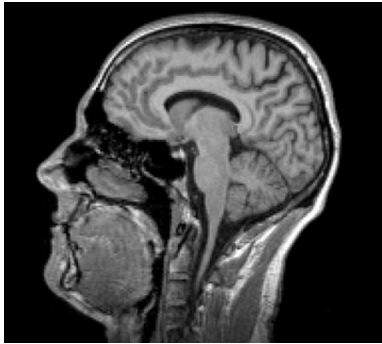


Image representation

Most classification algorithms require **input vectors**!

Image representation

Most classification algorithms require input vectors! There are many methods to convert an image into a vector. Here, we use the simplest method, which directly converts the image matrix into a vector.



Image

Convert a matrix into a column vector

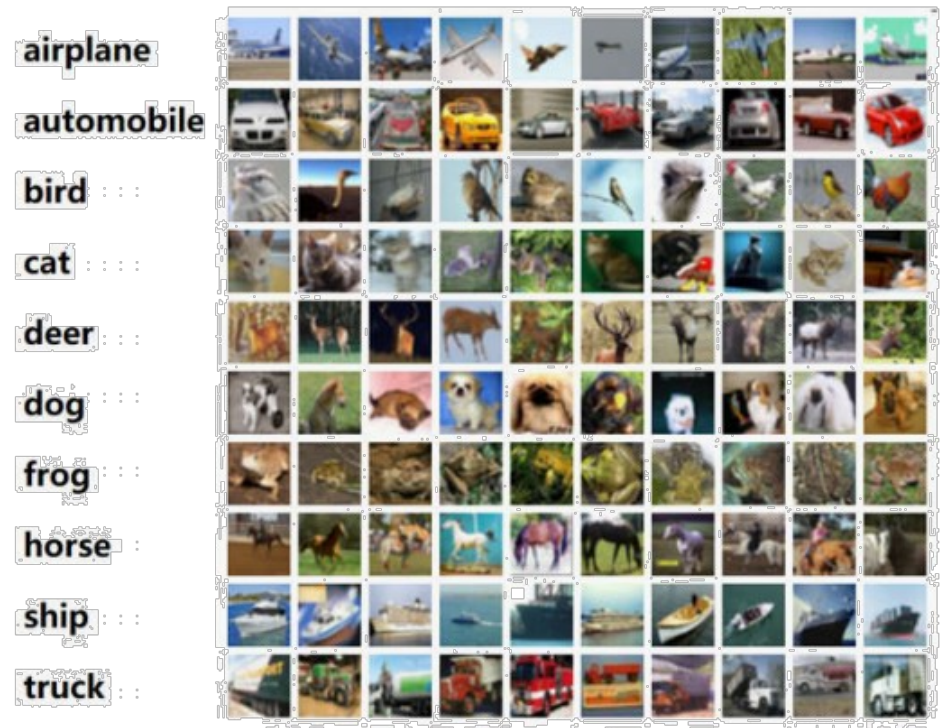


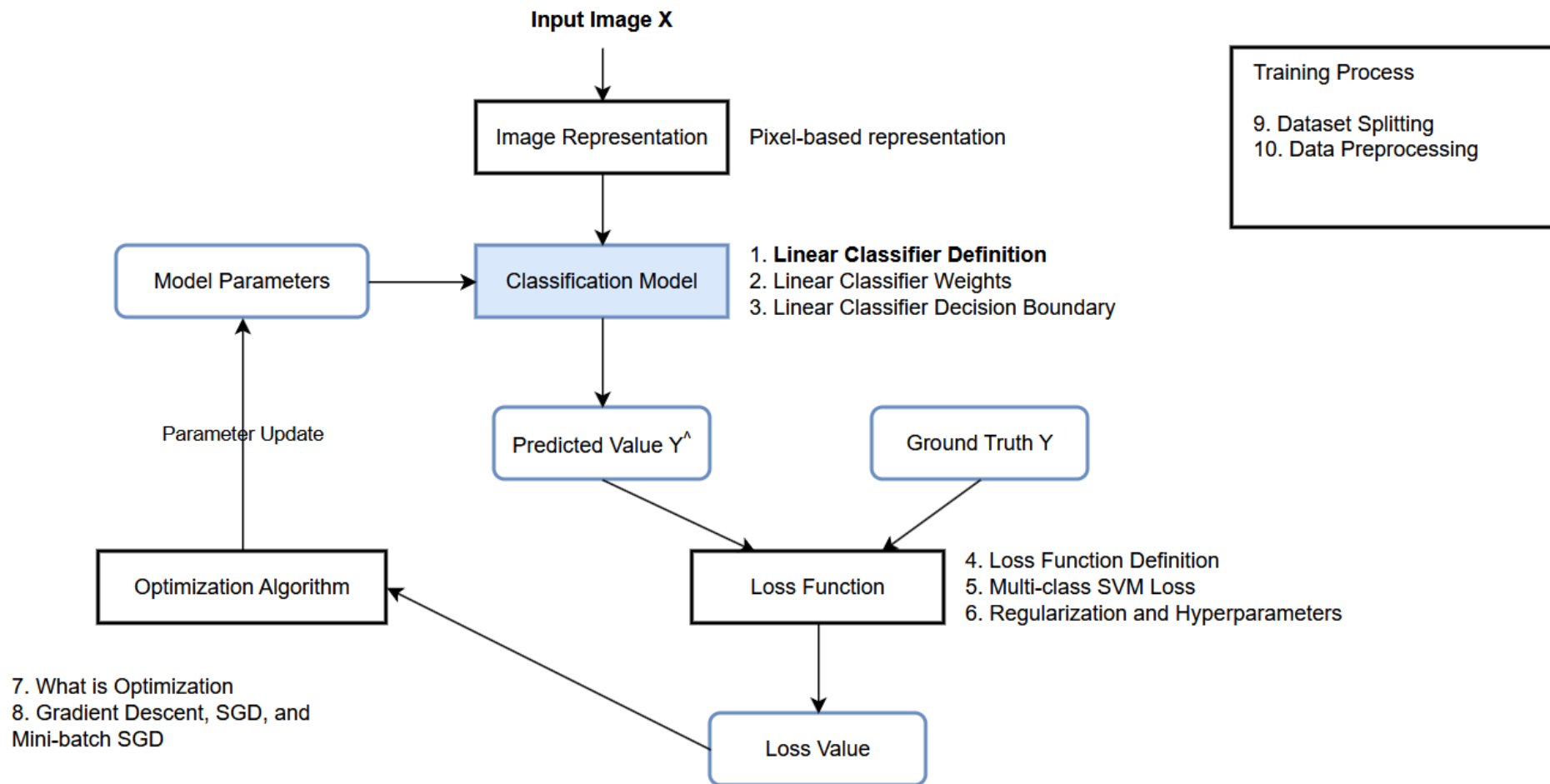
$$x = \begin{bmatrix} r_1 \\ g_1 \\ b_1 \\ \vdots \\ r_n \\ g_n \\ b_n \end{bmatrix}$$

Image representation

In CIFAR10, how many dimensions does the vector converted from each image have?

Answer: 3072 ($=32*32*3$)
dimensional column vector



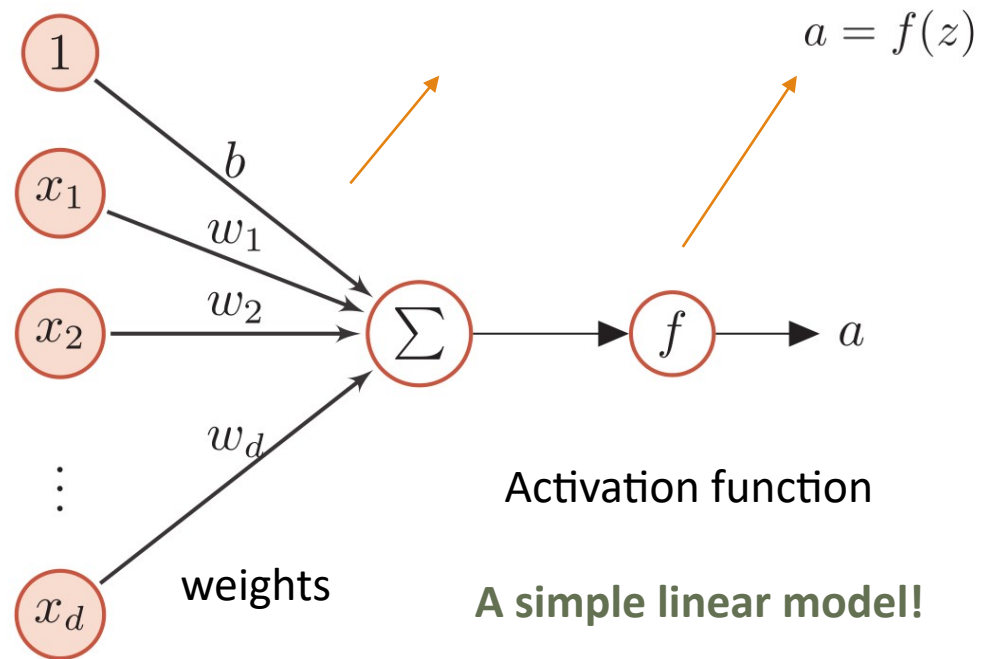
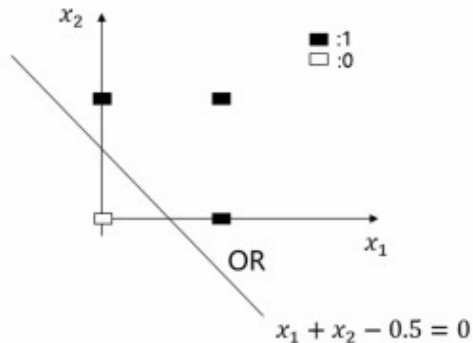


Why start with a linear classifier?

- Simple in form and easy to understand
- Through hierarchical structures (neural networks) or high-dimensional mapping (support vector machines), powerful nonlinear models can be formed.

Artificial neuron

- The perceptron is the simplest type of artificial neural network invented by Frank Rosenblatt in 1957.
- A single-layer perceptron is essentially a binary classifier.



Multi-class Classification

One-vs-Rest Method

$$\left. \begin{array}{lll} \text{Class 1} & \text{vs} & \text{Other Classes} \rightarrow \hat{f}_1(\mathbf{x}) \\ \text{Class 2} & \text{vs} & \text{Other Classes} \rightarrow \hat{f}_2(\mathbf{x}) \\ & \vdots & \\ \text{Class } c & \text{vs} & \text{Other Classes} \rightarrow \hat{f}_c(\mathbf{x}) \end{array} \right\} \hat{y} = \operatorname{argmax}_{y=1\dots c} \hat{f}_y(\mathbf{x})$$

For each class corresponding to $y = 1, 2, \dots, c$, set its label to $+1$, and for all remaining classes other than y , set their label to -1 .

When classifying a test sample $\mathbf{x}$, use the c discriminant functions obtained from the c binary classification problems

$$\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \dots, \hat{f}_c(\mathbf{x})$$

Predict on the training samples and calculate their function values. The predicted class $\hat{y}$ is the the corresponding class with the maximum function value. We have

$$\hat{y} = \operatorname{argmin}_{y=1\dots c} \hat{f}_y(\mathbf{x})$$

Multi-class Classification

Problem Background

We have a dataset where each sample is an image of a handwritten digit, and the label is a digit from 0 to 9. Our task is to train a model that can correctly classify an input image into one of the classes from 0, 1, 2, ..., 9.

Core Idea of the One-vs-Rest Method

The core idea of the One-vs-Rest method is to decompose a multi-class classification problem into multiple binary classification problems. For each class y , we train a classifier whose task is to determine whether the input sample belongs to class y . Specifically:

1. **Class y vs Other Classes:** Mark samples of class y as $+1$, and samples of all other classes as -1 .
2. For each class y (from 0 to 9), train a binary classifier, ultimately obtaining 10 classifiers $\hat{f}_0, \hat{f}_1, \dots, \hat{f}_9$.
3. During prediction, input the sample into all 10 classifiers, and choose the class corresponding to the classifier with the largest output value as the prediction result.

Multi-class Classification

Practical Example

Assume we have an input sample x , which is an image of a handwritten digit “3”. We wish to predict its class using the One-vs-Rest method.

1. Training Phase

- **Class 0 vs. Non-0:** Train a classifier $\hat{f}_0$ to determine if the input is "0".
- **Class 1 vs. Non-1:** Train a classifier $\hat{f}_1$ to determine if the input is "1".
- **Class 2 vs. Non-2:** Train a classifier $\hat{f}_2$ to determine if the input is "2".
- **Class 3 vs. Non-3:** Train a classifier $\hat{f}_3$ to determine if the input is "3".
- And so on, until all 10 classifiers are trained.

Multi-class Classification

2. Prediction Phase

Input the sample x (handwritten digit '3') into all 10 classifiers to obtain the output value of each classifier:

$$f_0(x), f_1(x), f_2(x), \dots, f_9(x)$$

Assuming these output values are as follows:

$$f_0(x) = -0.5,$$

$$f_1(x) = -0.3,$$

$$f_2(x) = -0.2,$$

$$f_3(x) = +0.8,$$

$$f_4(x) = -0.4,$$

$$f_5(x) = -0.1,$$

$$f_6(x) = -0.6,$$

$$f_7(x) = -0.7,$$

$$f_8(x) = -0.9,$$

$$f_9(x) = -0.2.$$

Multi-class Classification

3. Final Prediction

Choose the class corresponding to the classifier with the largest output value as the prediction result:

$$\hat{y} = \arg \max_{y=0,\dots,9} \hat{f}_y(x)$$

In this example, the classifier with the maximum output value is $\hat{f}_3(x) = +0.8$, so the prediction result is:

$$\hat{y} = 3$$

What is a linear classifier

A linear classifier is a linear mapping that maps input image features to class scores.

Linear Classifier Decision Making

Linear classifier for the i -th class:

$$f_i(\mathbf{x}, \mathbf{w}_i) = \mathbf{w}_i^T \mathbf{x} + b_i,$$

$$i = 1, \dots, c$$

Each class has its own parameters $\mathbf{w}$ and b

$\mathbf{x}$ represents the input d -dimensional image vector,
 c is the number of classes

$\mathbf{w}_i = [w_{i1} \ \dots \ w_{id}]^T$ is the weight vector for the i -th class, b_i is the bias

Decision rule:

If $f_i(\mathbf{x}) > f_j(\mathbf{x}), \ \forall j \neq i,$

then decide that the input image $\mathbf{x}$
belongs to the i -th class

Linear Classifier Example

Task: Assign a class label to the image (Car class, Cat class, Bird class)



Image

Linear Classifier

$$\mathbf{w}_i^T \mathbf{x} + b_i = f_i, i = 1, 2, 3$$

?

Linear Classifier Example

Image representation x

$$\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}$$

Linear Classifier Decision Steps:

1. Represent image as a vector



Linear Classifier Example

		Weights w_i^T	Image Representation $\mathbf{x}$	Bias b_i	Scores f_i
Car Class	w_1^T	0.2 -0.5 0.1 2.0	$\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}$	1.1 b_1	f_1 -97.9
Cat Class	w_2^T	1.5 1.3 2.1 0.0		3.2 b_2	f_2 434.7
Bird Class	w_3^T	0 0.25 0.25 -0.3		-1.2 b_3	f_3 63.15

Linear Classifier Decision Steps:

1. Represent the image as a vector
2. **Calculate** the scores for each class of the current image



Linear Classifier Example

		Weights w_i^T				Image Representation x		Bias b_i			Score f_i	
Car class	w_1^T	0.2	-0.5	0.1	2.0	$\begin{bmatrix} 56 \\ 231 \\ -24 \\ 2 \end{bmatrix}$	+	1.1	b_1	f_1	-97.9	=
Cat class	w_2^T	1.5	1.3	2.1	0.0			3.2	b_2	f_2	434.7	
Bird class	w_3^T	0	0.25	0.25	-0.3			-1.2	b_3	f_3	63.15	

Linear Classifier Decision Steps:

1. Represent image as vector
2. Calculate class scores for current image
3. Determine class based on scores



Cat class



Matrix Representation of Linear Classifier

	Weight matrix w_i^T	Image vector x	Bias vector b_i	Score vector f
Car Class w_1^T	0.2 -0.5 0.1 2.0	$\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}$	1.1 b_1	-97.9 f_1
Cat Class w_2^T	1.5 1.3 2.1 0.0		3.2 b_2	434.7 f_2
Bird Class w_3^T	0 0.25 0.25 -0.3		-1.2 b_3	63.15 f_3
	Weight Matrix W		Bias Vector b	Score Vector f



Matrix Representation of Linear Classifier:

$$f(x, W) = Wx + b$$

Where, x represents the input with dimension d .

x represents the input image, with dimension d .

f is the score vector, with dimension equal to the number of classes c .

$W = [w_1 \cdots w_c]^T$ is the weight matrix.

$w_i = [w_{i1} \cdots w_{id}]^T$ is the weight vector for the i -th class.

$b = [b_1 \cdots b_c]^T$ is the bias vector, and b_i is the bias for the i -th class.

Matrix Representation of Linear Classifiers

Problem: What are the dimensions of W , x , b for the classifier in the CIFAR10 dataset classification task?

Answer: CIFAR10 has 10 classes and image size is 32x32x3, therefore:

x is the image vector, its dimension is 3072;

W is the weight matrix, its dimension is 10x3072;

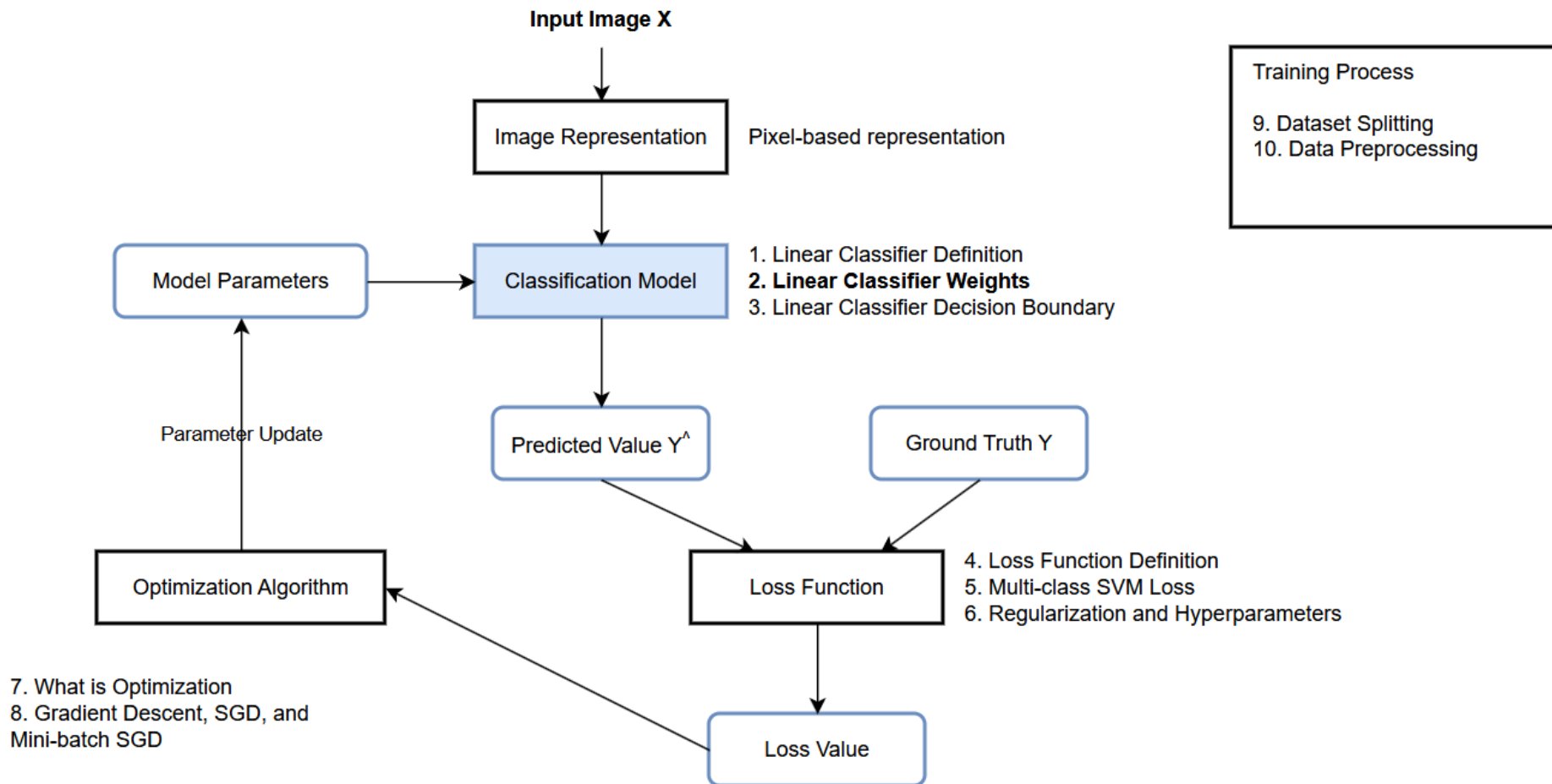
b is the bias vector, its dimension is 10x1;

f is the score vector, its dimension is 10x1;

Matrix Representation of a Linear Classifier:

$$f(x, W) = Wx + b$$

- Where x represents the input image, its dimension is d .
- f is the score vector, its dimension equals the number of classes c .
- $W = [w_1 \cdots w_c]^T$ is the weight matrix,
- $w_i = [w_{i1} \cdots w_{id}]^T$ is the weight vector for the i^{th} class,
- $b = [b_1 \cdots b_c]^T$ is the bias vector, b_i is the bias for the i -th class.



Linear Classifier Weight Vectors

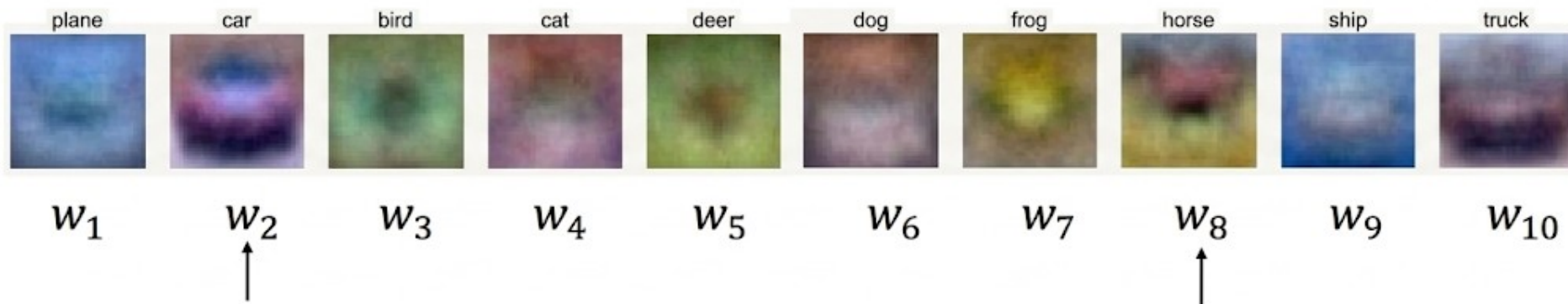
Linear classifier for the i -th class:

$$f_i(x, w_i) = w_i^T x + b_i,$$

$$i = 1, \dots, c$$

x represents the input d -dimensional image vector,
 c is the number of classes.

$w_i = [w_{i1} \ \dots \ w_{id}]^T$ is the weight vector for class i , b_i the bias.



The car template appears to
have memorized a red car

The horse template appears to
have memorized two heads

Linear Classifier Weight Vectors

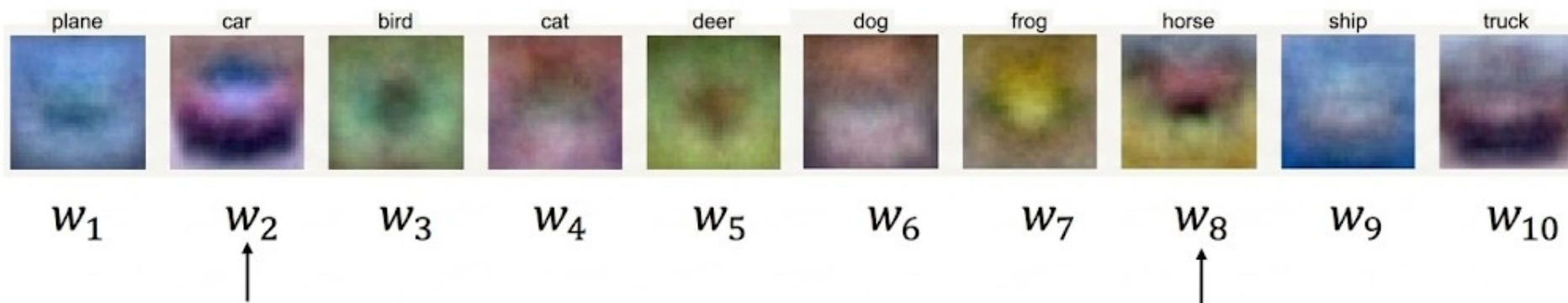
Linear classifier for the i -th class:

$$f_i(x, w_i) = w_i^T x + b_i,$$

$$i = 1, \dots, c$$

x represents the input d -dimensional image vector,
 c is the number of classes.

$w_i = [w_{i1} \ \dots \ w_{id}]^T$ is the weight vector for class i , b_i the bias.



The car template appears to have memorized a red car

↑
The horse template appears to have memorized two heads

Linear Classifier Weight Vectors

Linear classifier for the i -th class

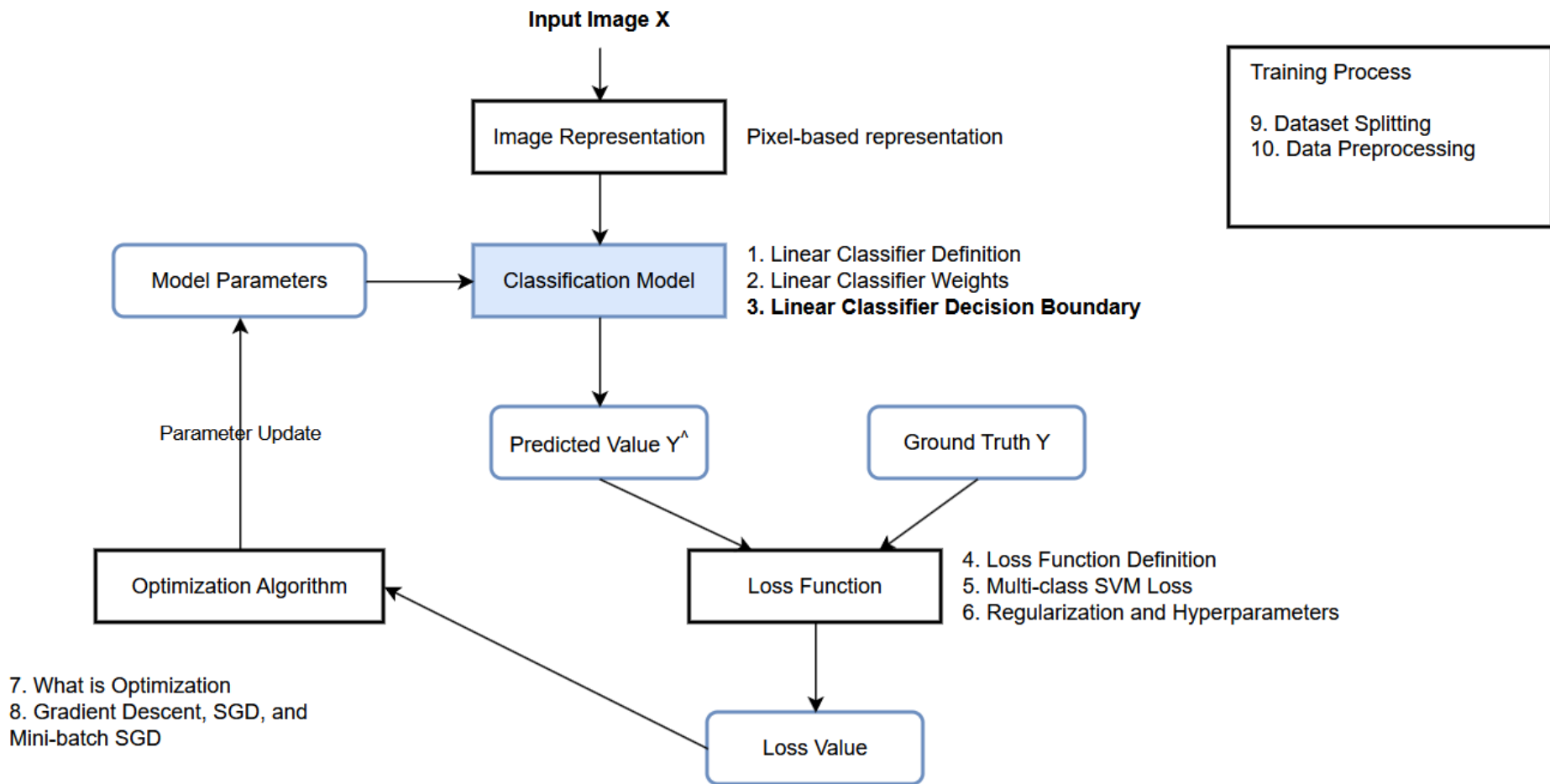
$$f_i(x, w_i) = w_i^T x + b_i,$$
$$i = 1, \dots, c$$

x represents the input d -dimensional image vector, c is the number of classes

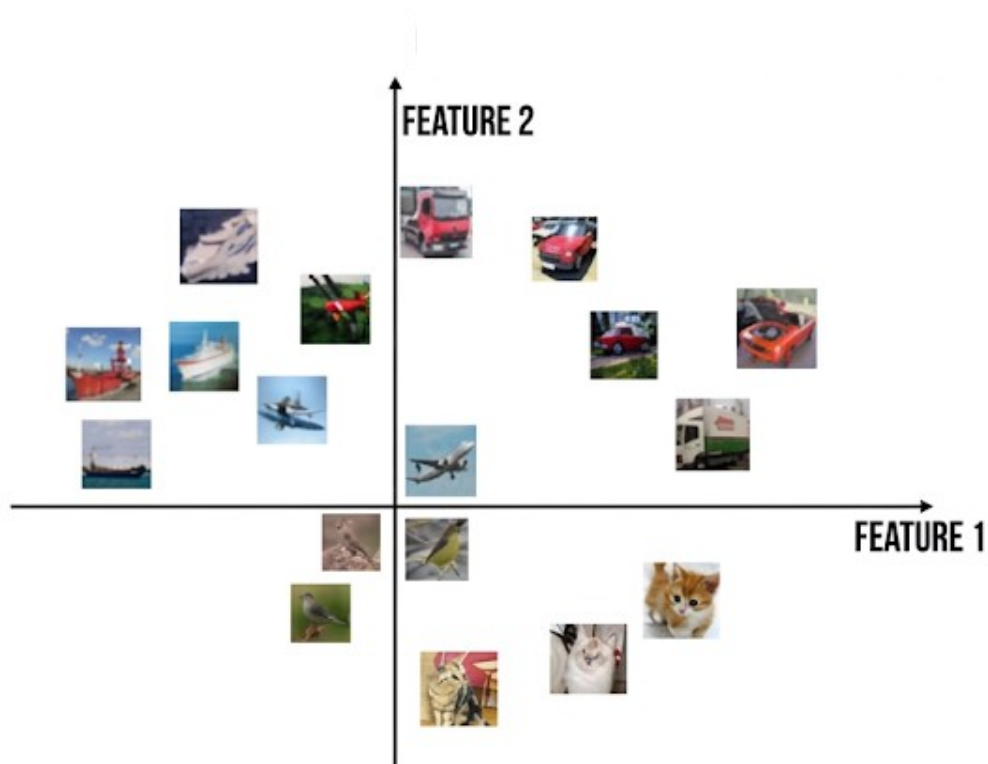
$w_i = [w_{i1} \ \dots \ w_{id}]^T$ is the weight vector for the i -th class, b_i is the bias



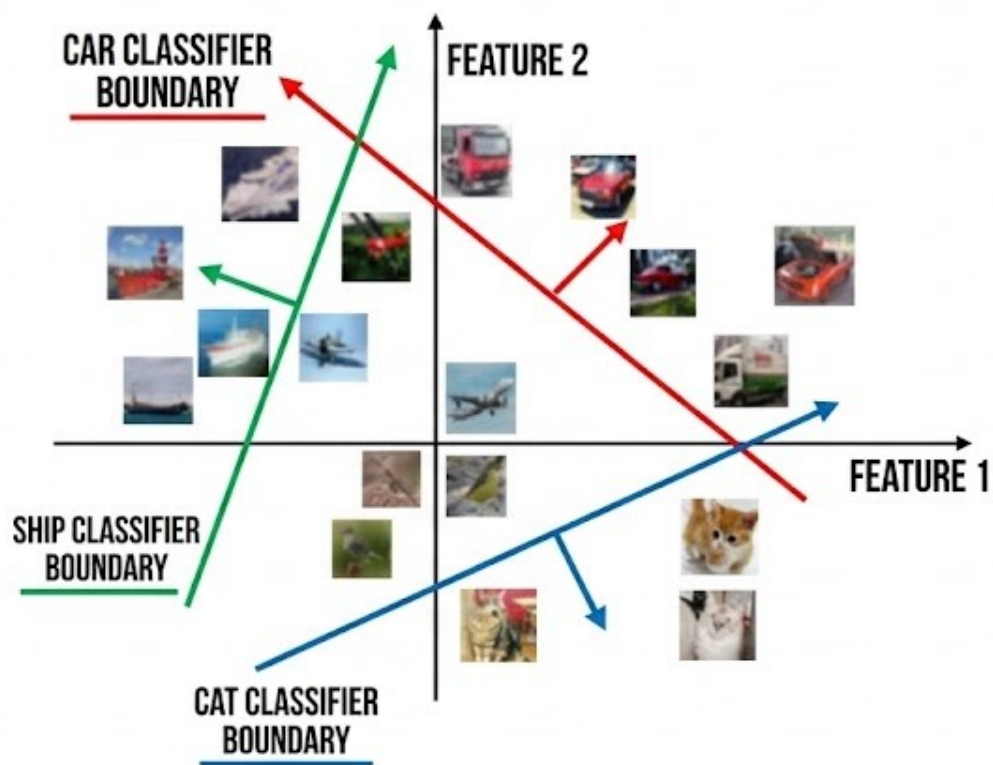
- Weights can be seen as a kind of template
- The higher the degree of match between the input image and the evaluation template, the higher the score output by the classifier



DECISION BOUNDARIES OF LINEAR CLASSIFIERS



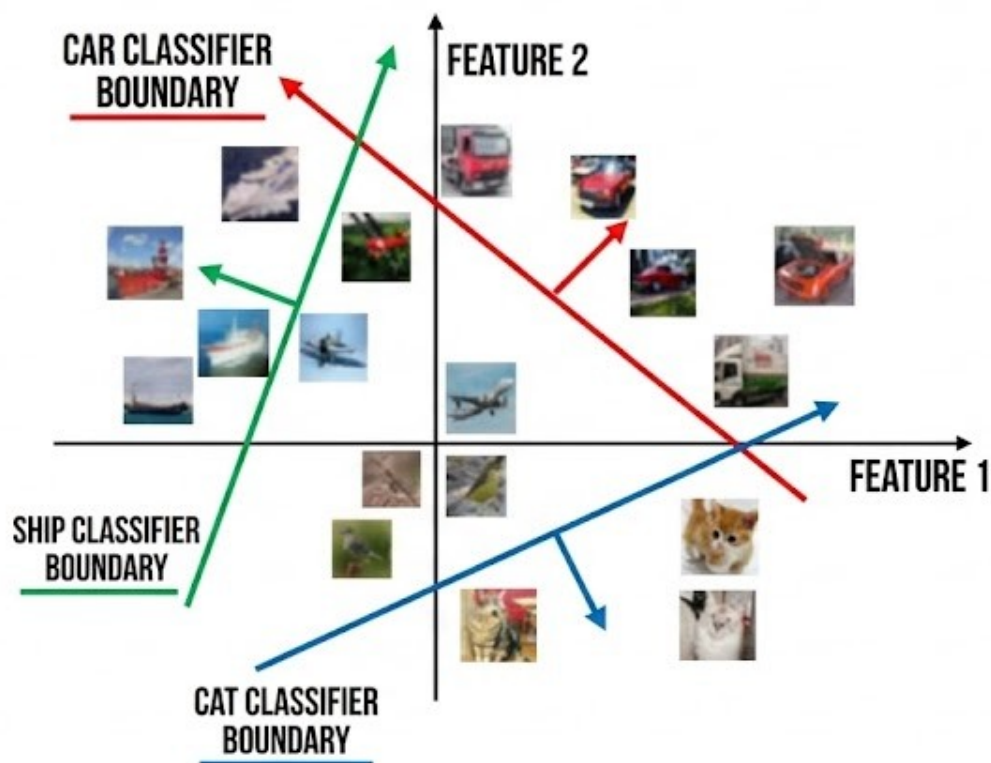
DECISION BOUNDARIES OF LINEAR CLASSIFIERS



- Lines with score = 0 are the decision boundaries.

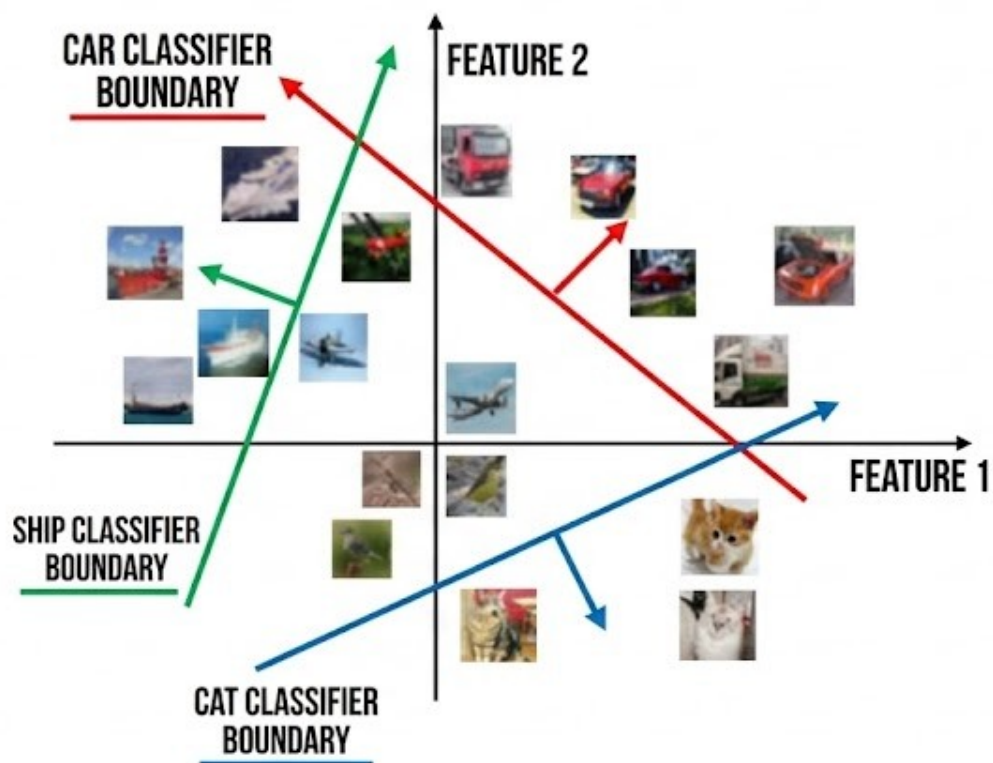
$$w_i^T x + b_i = 0, i = 1, \dots, c$$

DECISION BOUNDARIES OF LINEAR CLASSIFIERS



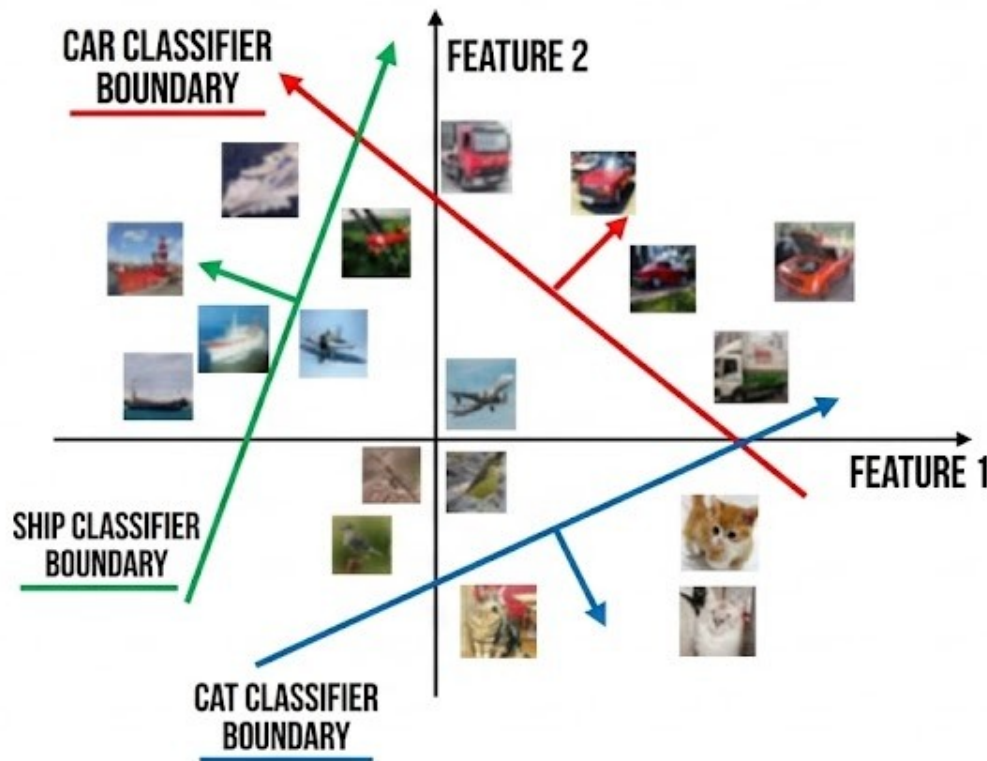
- Lines with score = 0 are the decision boundaries.
$$w_i^T x + b_i = 0, i = 1, \dots, c$$
- w controls the orientation of the line.

DECISION BOUNDARIES OF LINEAR CLASSIFIERS

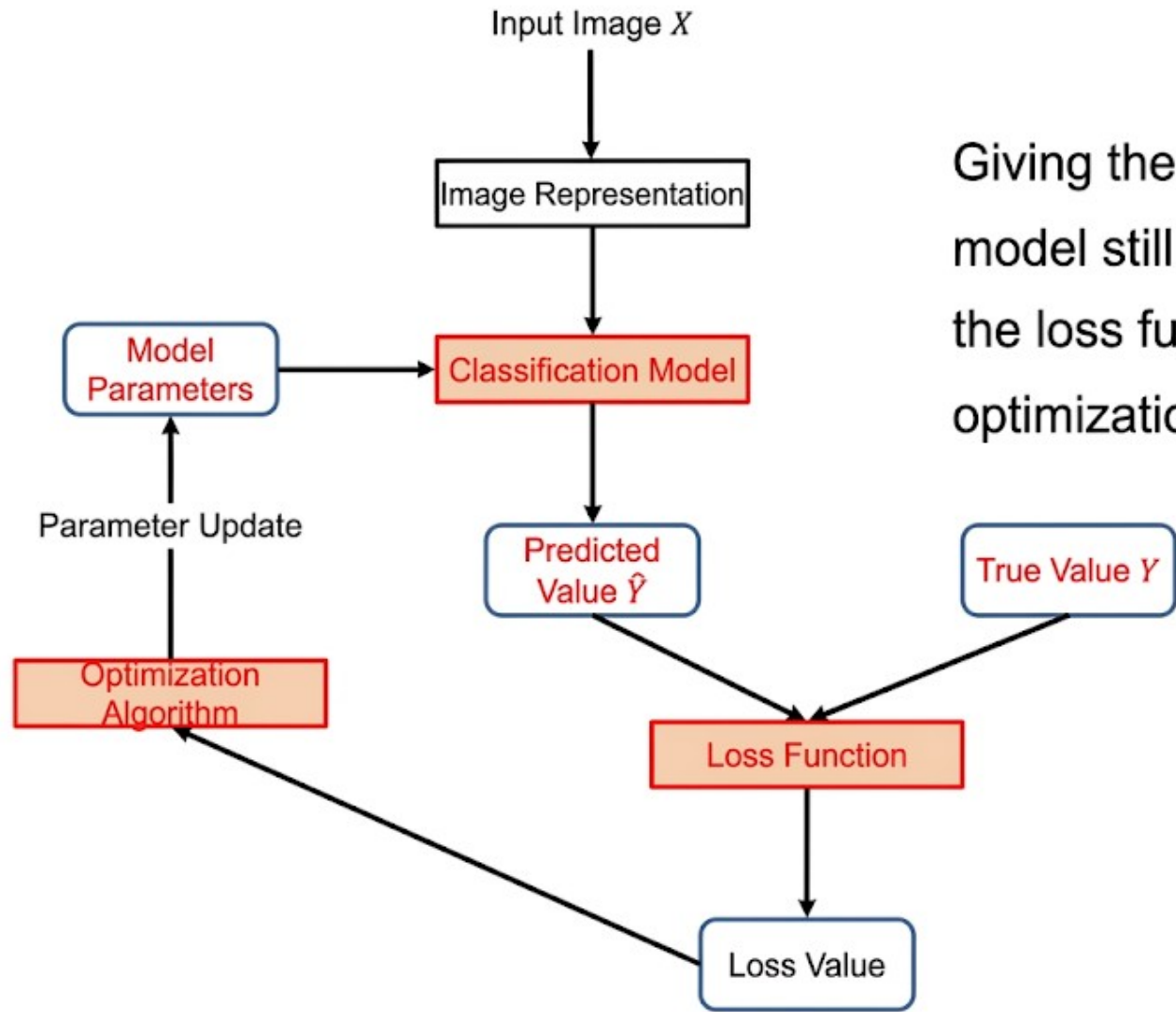


- Lines with score = 0 are the decision boundaries.
$$w_i^T x + b_i = 0, i = 1, \dots, c$$
- w controls the orientation of the line.
- b controls the bias of the line.

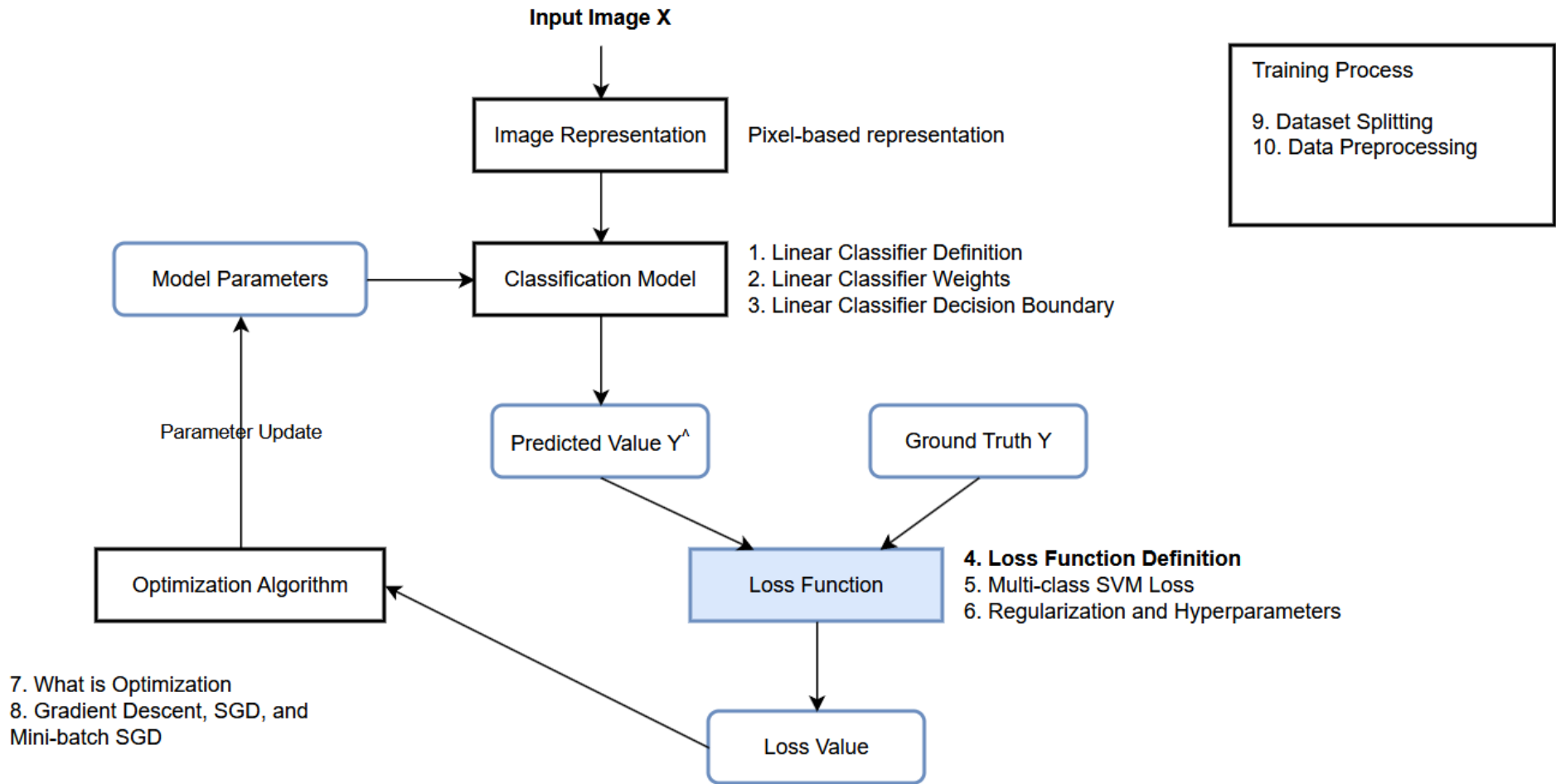
DECISION BOUNDARIES OF LINEAR CLASSIFIERS



- Lines with score = 0 are the decision boundaries.
$$w_i^T x + b_i = 0, i = 1, \dots, c$$
- w controls the orientation of the line.
- b controls the bias of the line.
- Arrow direction represents the classifier's positive direction. Along the arrow direction, moving further from the boundary gives higher scores.



Giving the optimal classification model still requires the help of the loss function and optimization algorithm.



How to measure the performance of a classifier on the current sample?

Classifier 1

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.25	-0.3

$W1$

$$\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix} + \begin{bmatrix} 1.1 \\ 3.2 \\ -1.2 \end{bmatrix} = \begin{bmatrix} -97.9 \\ 434.7 \\ 63.15 \end{bmatrix}$$

$b1$ f

Car Class Score
Cat Class Score
Ship Class Score

✓



How to measure the performance of a classifier on the current sample?

Classifier 2

0.2	-0.1	0.1	1.7
1.5	0.2	2.1	5.0
-1.2	1.5	0.25	0.5

W_2

$$\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix} + \begin{bmatrix} 2.1 \\ -0.2 \\ 2.4 \end{bmatrix} = \begin{bmatrix} -6.1 \\ 190.6 \\ 286.3 \end{bmatrix}$$

b_2 f

Car Class Score
Cat Class Score
Ship Class Score

A blue arrow points from a small image of a kitten below to the value 2 in the vector $\begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}$. A large red 'X' is placed over the 'Ship Class Score'.



Loss Function

For the example sample, which classifier (Classifier 1 or Classifier 2) has a better effect?

How to quantitatively represent it?

Need a loss function to help

What is a Loss Function?

The loss function builds a bridge between model performance and model parameters, guiding the optimization of model parameters.

- A loss function is a function used to measure the degree of inconsistency between the predicted value of a given classifier and the true value, and its output is usually a non-negative real value.
- Its output non-negative real value can serve as a feedback signal to adjust the classifier parameters, so as to reduce the loss value corresponding to the current instance and improve the classification performance of the classifier.

What is a Loss Function?

General Definition of a Loss Function:

$$L = \frac{1}{N} \sum_i L_i (f(x_i, W), y_i)$$

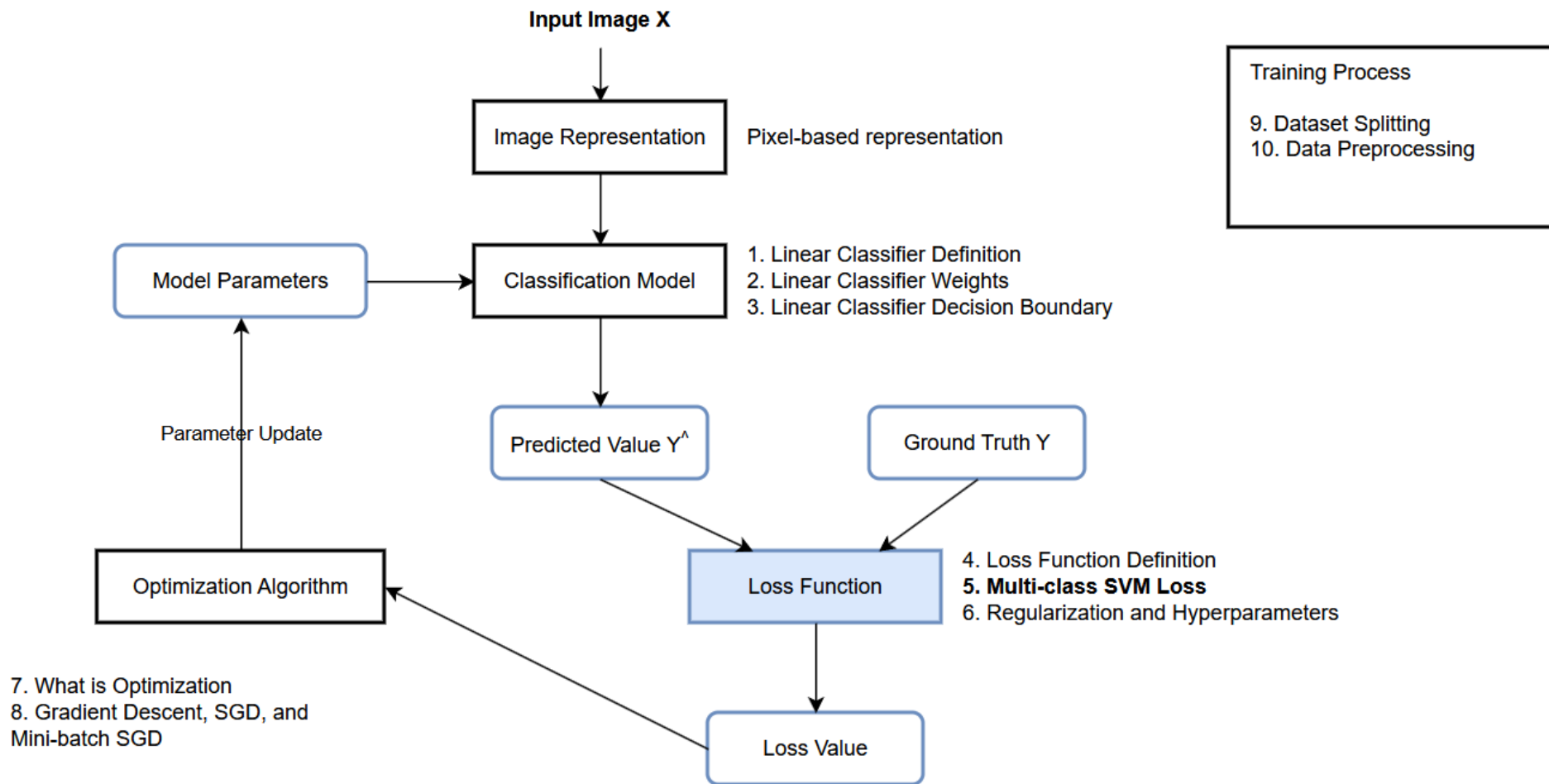
x_i represents the 'i'-th image in the dataset;

$f(x_i, W)$ is the classifier's class prediction for ' x_i ';

y_i is the true class label for sample 'i' (integer);

L_i is the loss for the 'i'-th sample given the predicted value;

L is the dataset loss, which is the average of the loss of all samples in the dataset.



Multiclass Support Vector Machine Loss

$$s_{ij} = f_j(x_i, w_j, b_j) = w_j^T x_i + b_j$$

- j — class label, values in range $\{1, 2, \dots, C\}$
- w_j, b_j — parameters for the j -th class classifier
- x_i — represents the i -th sample in the dataset
- s_{ij} — predicted score of the j -th class for the
- s_{y_i} — predicted score of the true class for the i -th sample

The Multiclass SVM Loss for the i -th sample is defined as follows:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_{ij} + 1 \\ s_{ij} - s_{y_i} + 1 & \text{otherwise} \end{cases}$$
$$= \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

Multiclass Support Vector Machine Loss

$$s_{ij} = f_j(x_i, w_j, b_j) = w_j^T x_i + b_j$$

j — class label, values in $\{1, 2, \dots, C\}$

w_j, b_j — parameters of the j -th class

x_i — represents the i -th sample in

s_{ij} — predicted score of the j -th class

s_{y_i} — predicted score of the true class

The Multiclass SVM Loss for the i -th sample is defined as follows:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_{ij} + 1 \\ s_{ij} - s_{y_i} + 1 & \text{otherwise} \end{cases}$$
$$= \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

◆ If the score of the correct class is 1 point higher than that of an incorrect class,

Multiclass Support Vector Machine Loss

$$s_{ij} = f_j(x_i, w_j, b_j) = w_j^T x_i + b_j$$

j — class label, values in $\{1, 2, \dots, C\}$

w_j, b_j — parameters of the j -th class

x_i — represents the i -th sample in

s_{ij} — predicted score of the j -th class

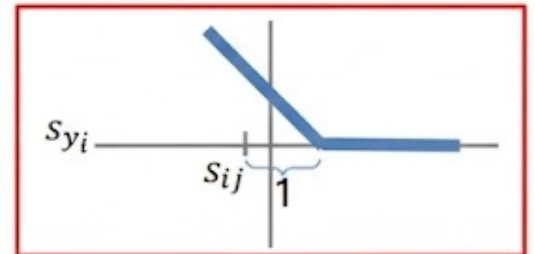
s_{y_i} — predicted score of the true class

The Multiclass SVM Loss for the i -th sample is defined as follows:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_{ij} + 1 \\ s_{ij} - s_{y_i} + 1 & \text{otherwise} \end{cases}$$
$$= \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

- ◆ If the score of the correct class is 1 point higher than that of an incorrect class,
- ◆ Otherwise, a loss occurs

Multiclass Support Vector Machine Loss



$$s_{ij} = f_j(x_i, w_j, b_j) = w_j^T x_i + b_j$$

The Multiclass SVM Loss for the i -th sample is defined as follows:

j — class label, values in $\{1, 2, \dots, C\}$

w_j, b_j — parameters of the j -th class

x_i — represents the i -th sample in

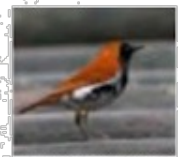
s_{ij} — predicted score of the j -th class

s_{y_i} — predicted score of the true class

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_{ij} + 1 \\ s_{ij} - s_{y_i} + 1 & \text{otherwise} \end{cases}$$
$$= \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

- ◆ If the score of the correct class is 1 point higher than that of an incorrect class,
- ◆ Otherwise, a loss occurs $\max(0, \cdot)$ loss — often called hinge loss (hingeloss)

Example: Assume we have one training sample for each of the 3 classes. The classifier is a linear classifier $f(x, W) = Wx + b$, where the weights W and b are known. The classifier's scoring for the three samples is as follows:



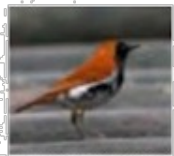
bird	cat	car	loss
0.6	-2.3	1.9	2.3
1.7	2.9	2.3	0.4
3.1	-2.6	4.3	0

Classifier loss for this bird image:

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

$$\begin{aligned} &= \max(0, -2.3 - 0.6 + 1) \\ &\quad + \max(0, 1.9 - 0.6 + 1) \\ &= \max(0, -1.9) + \max(0, 2.3) \\ &= 0 + 2.3 \\ &= 2.3 \end{aligned}$$

Example: Assume we have one training sample for each of the 3 classes. The classifier is a linear classifier $f(x, W) = Wx + b$, where the weights W and b are known. The classifier's scoring for the three samples is as follows:



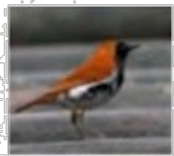
bird	cat	car	loss
0.6	-2.3	1.9	2.3
1.7	2.9	2.3	0.4
3.1	-2.6	4.3	0

Classifier loss for this bird image:

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

$$\begin{aligned} &= \max(0, 1.7 - 2.9 + 1) \\ &\quad + \max(0, 2.3 - 2.9 + 1) \\ &= \max(0, -0.2) + \max(0, 0.4) \\ &= 0 + 0.4 \\ &= 0.4 \end{aligned}$$

Example: Assume we have one training sample for each of the 3 classes. The classifier is a linear classifier $f(x, W) = Wx + b$, where the weights W and b are known. The classifier's scoring for the three samples is as follows:



bird	cat	car	loss
0.6	-2.3	1.9	2.3
1.7	2.9	2.3	0.4
3.1	-2.6	4.3	0

Classifier loss for this bird image:

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

$$\begin{aligned} &= \max(0, 3.1 - 4.3 + 1) \\ &\quad + \max(0, -2.6 - 4.3 + 1) \\ &= 0 \end{aligned}$$

Example: Must have 3 shares on inaining hampes, , not classiifier $f(x, W) = Wx + b$, it mights W , b and cnown. The pasnitinearing the classifier numbriess as below:



bird	cat	car	loss
0.6	-2.3	1.9	2.3
1.7	2.9	2.3	0.4
3.1	-2.6	4.3	0

Current classifier on each dataset's lolss:

$$L = \frac{1}{N} \sum_{i=1}^N L_i \quad (\text{each datass of averge lolss})$$

$$L = (2.3 + 0.4 + 0)/3 = 0.9$$

Problem Q&A

➤ Loss Function:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

➤ Multiclass SVM Loss for a Single Sample:

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

➤ Linear Classifier:

$$s_{ij} = w_j^T x_i + b_j$$

1: What are the maximum and minimum values of the multiclass SVM loss L_i ?

2: If w and b are very small during initialization, what will the loss L be?

3: What changes would occur to the loss L_i if all categories are considered (including $j = y_i$)?

4: If the sum replaces the average during total loss L calculation?

5: If $L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)^2$ is used?

Revisiting Loss Functions

$$L(W) = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Question: Assuming a W exists such the loss function $L=0$, is this W unique?

Revisiting Loss Functions

Example: In two classifiers $f_1(x, W_1) = W_1 x$, $f_2(x, W_2) = W_2 x$, where $W_2 = 2 W_1$. For the input, which is known to be a car (as moon), classifier 2 is of gray, and the classifier classifies both not a car. Compute the multiclass SVM loss for this single sample:



	Bird	Cat	Car	Loss
Classifier 1	3.1	-2.6	4.3	?
Classifier 2	6.2	-5.2	8.6	?

Single Sample

Multi-class SVM loss for sample i :

$$L_i = \sum_{j \neq y_i} \max(0, s_{ij} - s_{y_i} + 1)$$

Classifier 1 Loss:

$$\begin{aligned}
 &= \max(0, 3.1 - 4.3 + 1) \\
 &+ \max(0, -2.6 - 4.3 + 1) \\
 &= \max(0, -0.2) + \max(0, -5.9) \\
 &= 0
 \end{aligned}$$

Classifier 2 Loss:

$$\begin{aligned}
 &= \max(0, 6.2 - 8.6 + 1) \\
 &+ \max(0, -5.2 - 8.6 + 1) \\
 &= \max(0, -1.4) + \max(0, -12.8) \\
 &= 0
 \end{aligned}$$

Revisiting Loss Functions

$$L(W) = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Question: Assuming a W exists such that the loss function $L=0$, is this W unique?

Answer: Not unique, because W_2 also has $L = 0$

How should we choose between W_1 and W_2 ?

Regularization Term Loss

$$L(W) = \underbrace{\frac{1}{N} \sum_i L_i(f(x_i, W), y_i)}_{\text{Data Loss: Model predictions need to fit the training set}} + \underbrace{\lambda R(W)}_{\text{Regularization Loss: Prevent the model from learning "too well" on the training set.}}$$

Data Loss: Model predictions need to fit the training set

Regularization Loss: Prevent the model from learning "too well" on the training set.

- ◆ $R(W)$ is a function relads to the weigrt and image to the unrelated
- ◆ λ is a hyperparameter controls the regulariza-tion loss in the total loss

What are Hyperparameters?

- Parameters whose values are set before the learning process begins, not learned.
- Hyperparameters generally have a significant impact on model performance.

$$L(W) = \underbrace{\frac{1}{N} \sum_i L_i(f(x_i, W), y_i)}_{\text{Data Loss: Model predictions need to match the training set}} + \underbrace{\lambda R(W)}_{\text{Regularization Loss: Prevents the model from learning "too well" on the training set.}}$$

Data Loss: Model predictions need to match the training set

Regularization Loss: Prevents the model from learning "too well" on the training set.

- $\lambda = 0$ Optimization result is only related to data loss.
- $\lambda = \infty$ Optimization result is unrelated to data loss, only considers weight loss.
At this point, the optimal solution is $W = 0$.

L2 Regularization Term

$$L(W) = \underbrace{\frac{1}{N} \sum_i L_i(f(x_i, W), y_i)}_{\text{Data Loss: The model prediction should match the training set}} + \underbrace{\lambda R(W)}_{\text{Regularization Loss: Prevents the model from learning 'too well' on the training set.}}$$

Data Loss: The model prediction should match the training set

Regularization Loss: Prevents the model from learning 'too well' on the training set.

L2 Regularization Term $R(W) = \sum_k \sum_l W_{k,l}^2$

L2 Regularization Loss penalizes large weight values, prefers dispersed weights, and encourages the classifier to use features from all dimensions, rather than heavily relying on a few dimensions.

L2 Loss Example

Example: $x = [1, 1, 1, 1]$

Classifier 1: $w_1 = [1, 0, 0, 0]$

Classifier 2: $w_2 = [0.25, 0.25, 0.25, 0.25]$

Classifier Output:

$$w_1^T x = w_2^T x = 1$$

Regularization Loss:

$$R(w_1) = 1 \quad R(w_2) = 0.25$$

w_2 total loss is smaller

The regularization term gives the model a preference!!!

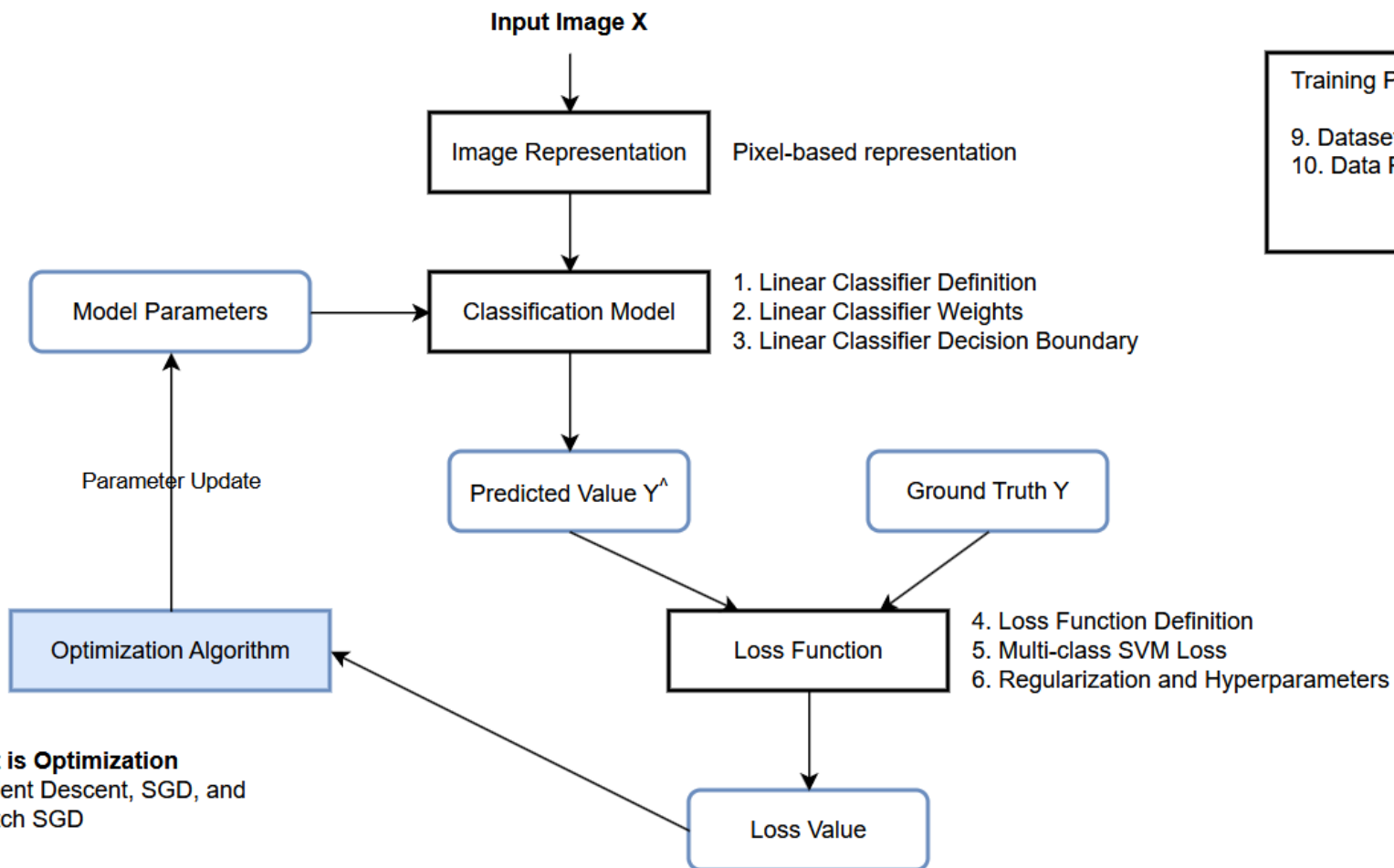
Common Regularization Term Losses

$$L(W) = \underbrace{\frac{1}{N} \sum_i L_i(f(x_i, W), y_i)}_{\text{Data Loss: Model predictions must match the training set}} + \underbrace{\lambda R(W)}_{\text{Regularization Loss: Prevents the model from learning "too well" on the training set.}}$$

L1 Regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

L2 Regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

Elastic net(L1+L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$



Training Process

- 9. Dataset Splitting
- 10. Data Preprocessing

7. What is Optimization

- 8. Gradient Descent, SGD, and Mini-batch SGD

What is Parameter Optimization?

Parameter optimization is one of the core steps of machine learning. It uses the output values of the loss function as feedback signals to adjust classifier parameters, to improve the classifier's predictive performance on training samples.

Optimization Algorithm Goal

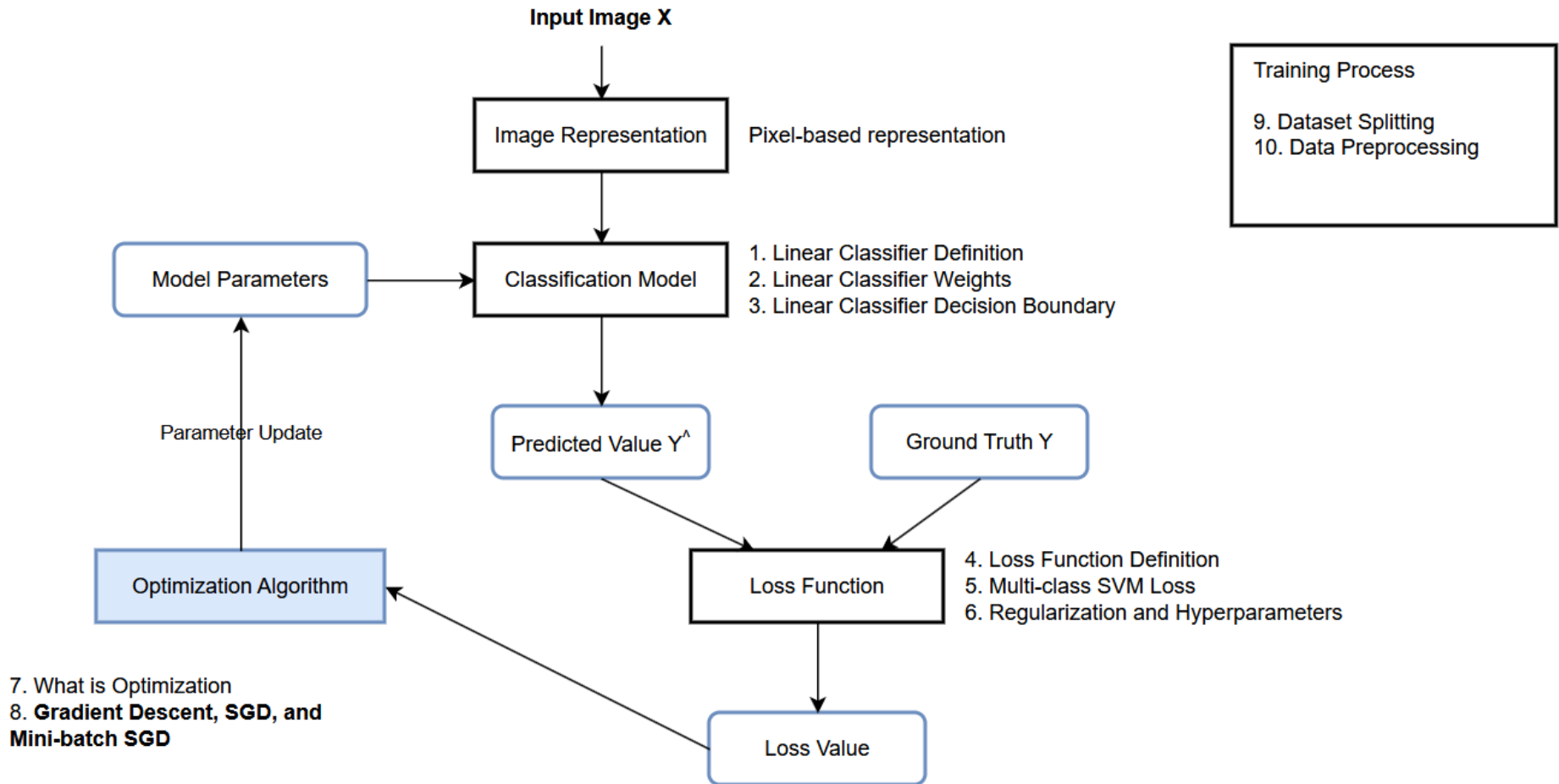
Loss Function
$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W)$$

The loss function L is a function of the parameter W . The objective of optimization is to find the set of parameters W that makes the loss function L optimal.

Direct Method:

$$\frac{\partial L}{\partial W} = 0$$

Typically, the form of L is quite complex, making it very difficult to solve for W directly from this equation!



Gradient Descent Algorithm

A Simple and Efficient Iterative Optimization Method!

Gradient Descent Algorithm

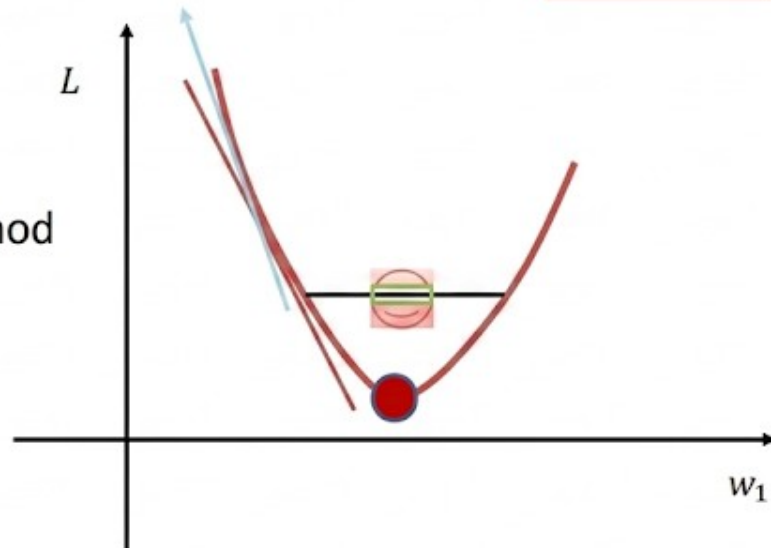
$$L(W) = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i) + \lambda R(W)$$

➤ Which way to go?

Ans: Negative gradient method

➤ How far to go?

Ans: Step size decides



Gradient Descent Algorithm

$$L(W) = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i) + \lambda R(W)$$

➤ Where to go?

Ans: Negative Gradient Method

➤ How far to go?

Ans: Step Size Decides



Gradient Descent Algorithm

Gradient Descent: Using all samples to calculate loss and update gradients

```
while True
```

```
    weights_gradient ← Calculate_gradient(loss, training_samples, weights)
```

```
    weights ← weights - learning_rate * weights_gradient
```

Updated
weights

Current
weights

Gradient Calculation

1. Numerical Method Computationally heavy, and imprecise!

One-dimensional variable, function derivation:

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

Example: Loss function $L(w) = w^2$, find the gradient at $w = 1$

$$\frac{dL(w)}{dw} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h} \approx \frac{L(1 + 0.0001) - L(1)}{0.0001} = 2.0001$$

Gradient Calculation

2. Analytical Method Accurate, Fast, Error-prone derivative derivation!

Example: Loss function $L(w) = w^2$, find the gradient at $w = 1$

$$\nabla L(w) = 2w$$

$$\nabla_{w=1} L(w) = 2$$



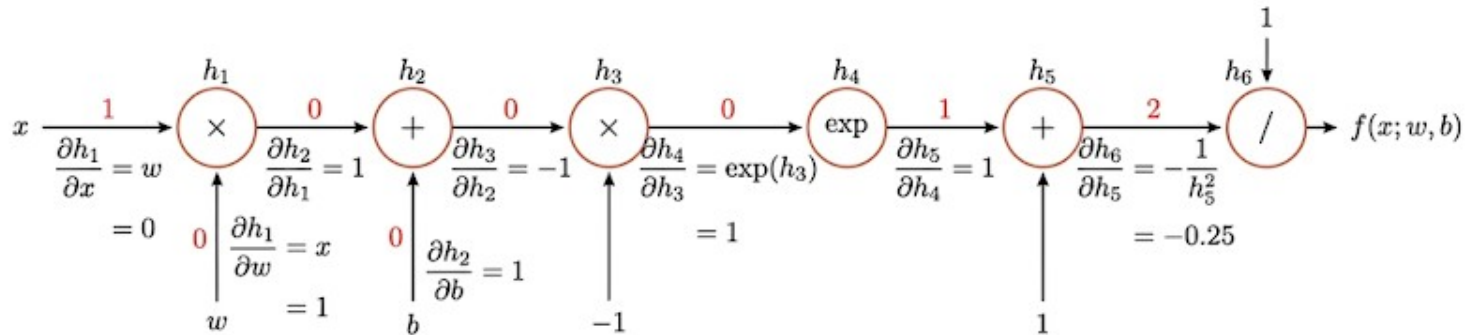
Gradient Calculation Summary

- Numerical Gradient: Approximate, slow, easy to write
- Analytical Gradient: Precise, fast, easy to err

What is the function of the numerical gradient?

Ans: Generally use analytical gradient for calculating gradient, while numerical gradient is **mainly** used for checking the correctness of analytical gradient (gradient checking).

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}$$



- When $x = 1, w = 0, b = 0$, we can get

$$\begin{aligned} \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25. \end{aligned}$$

Function	Derivative	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

Computational Efficiency of Gradient Descent Algorithms

Gradient Descent: Use all samples to calculate loss and update gradients

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$
$$\nabla_w L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_w L_i(x_i, y_i, W) + \lambda \nabla_w R(W)$$

while True

Weight Gradient $\leftarrow$ calculate_gradient(loss, training_samples, weights)

Weights $\leftarrow$ Weights - Learning_Rate * Weight_Gradient



When N is very large, weight gradient computation is huge!

Stochastic Gradient Descent Algorithm

Stochastic Gradient Descent: Randomly select one sample x_i each time, calculate loss, and update gradients

$$L(W) = L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

```
while True
```

```
  Data ← Sample from training data (training data, 1)
```

```
  Weight Gradient ← Calculate Gradient ( Loss, Data, Weights)
```

```
  Weights ← Weights - Learning_Rate * Weight_Gradient
```



Training on a single sample may introduce a lot of noise, and not every iteration moves toward the overall optimization direction.

Mini-batch Gradient Descent Algorithm

Mini-batch Stochastic Gradient Descent: Randomly select m samples each time each time (batch-size), calculate loss and update gradients

Hyperparameter: m (batch size)

$$L(W) = \frac{1}{m} \sum_{i=1}^m L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{m} \sum_{i=1}^m \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

while True

Weight Gradient $\leftarrow$ calculate_gradient(loss, training_samples)

Data $\leftarrow$ Sample from training data (training data, **batch_size**)

Weight Gradient $\leftarrow$ Calculate Gradient (Loss, **Data**, Weights)

Weight $\leftarrow$ Weight - Learning Rate * Weight Gradient

- iteration: represents 1 iteration, updating the weights once;
- batch-size: 1 iteration used or of samples;
- epoch: 1 epoch passing through in the training set or sapce.



tip: usually use a power of 2 as the batch size, such as selecting 32, 64, or 128 samples each time

Summary

Gradient Descent Method

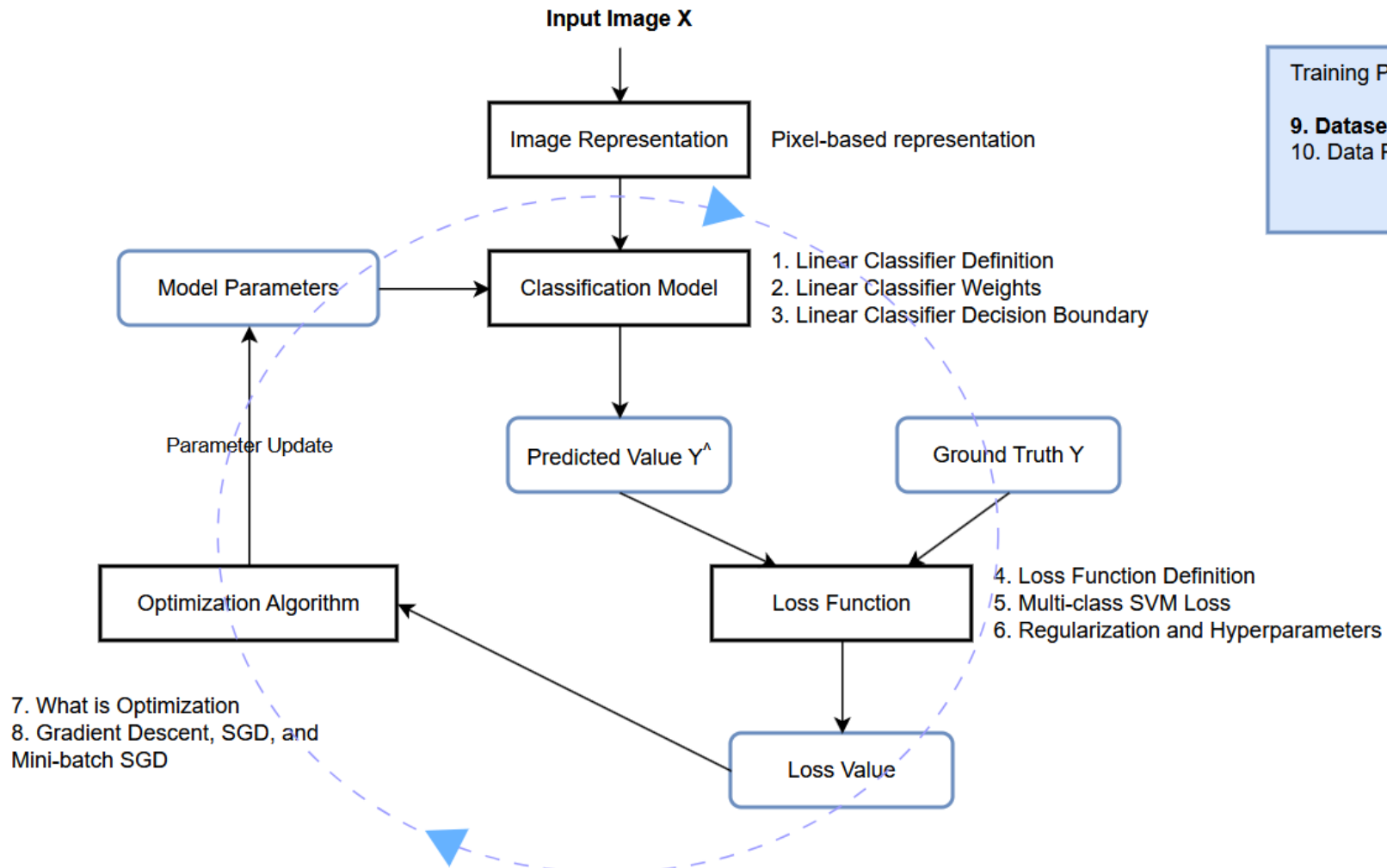
```
while True
    Weight Gradient  $\leftarrow$  calculate_gradient(loss, training_data, weights)
    Weights  $\leftarrow$  Weights - Learning_Rate * Weight_Gradient
```

Stochastic Gradient Descent Algorithm

```
while True
    Data  $\leftarrow$  sample_from_training_data(training_data, 1)
    Weight Gradient  $\leftarrow$  calculate_gradient(loss, Data, weights)
    Weights  $\leftarrow$  Weights - Learning_Rate * Weight_Gradient
```

Mini-batch Gradient Descent Algorithm

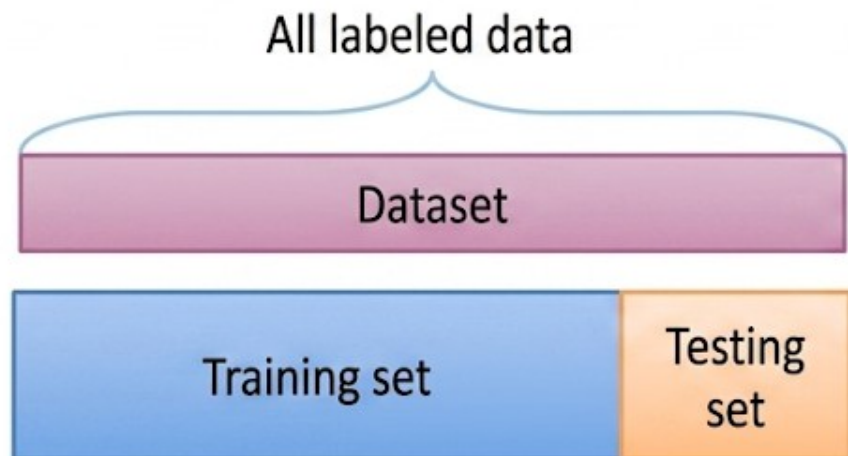
```
while True
    Mini-batch Data  $\leftarrow$  sample_from_training_data(training_data, batch_size)
    Weight Gradient  $\leftarrow$  calculate_gradient(loss, Mini-batch Data, weights)
    Weights  $\leftarrow$  Weights - Learning_Rate * Weight_Gradient
```



Training Process

9. Dataset Splitting
10. Data Preprocessing

Dataset Splitting



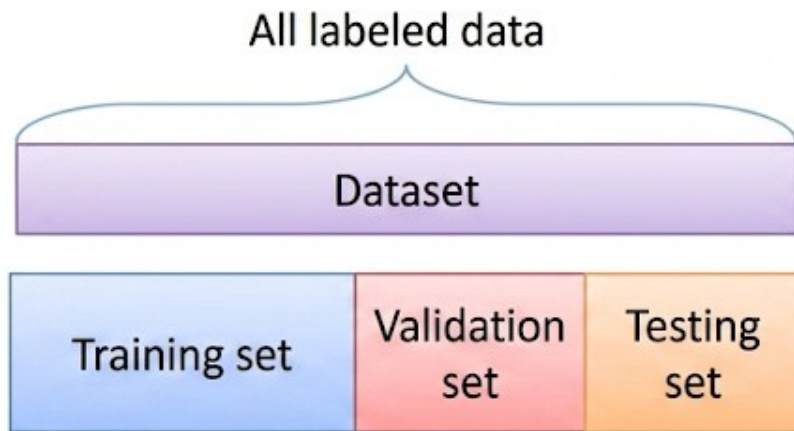
Problem: If the model contains hyperparameters (e.g., regularization strength), how to find the hyperparameters with the best generalization ability?

Answer: Use a Validation set

Tune on training set, select on training set?
select models on testing set?



Dataset Splitting



- Training set is used for learning classifier parameters with given hyperparameters;
- Validation set is used to select hyperparameters;
- Testing set evaluates generalization ability;

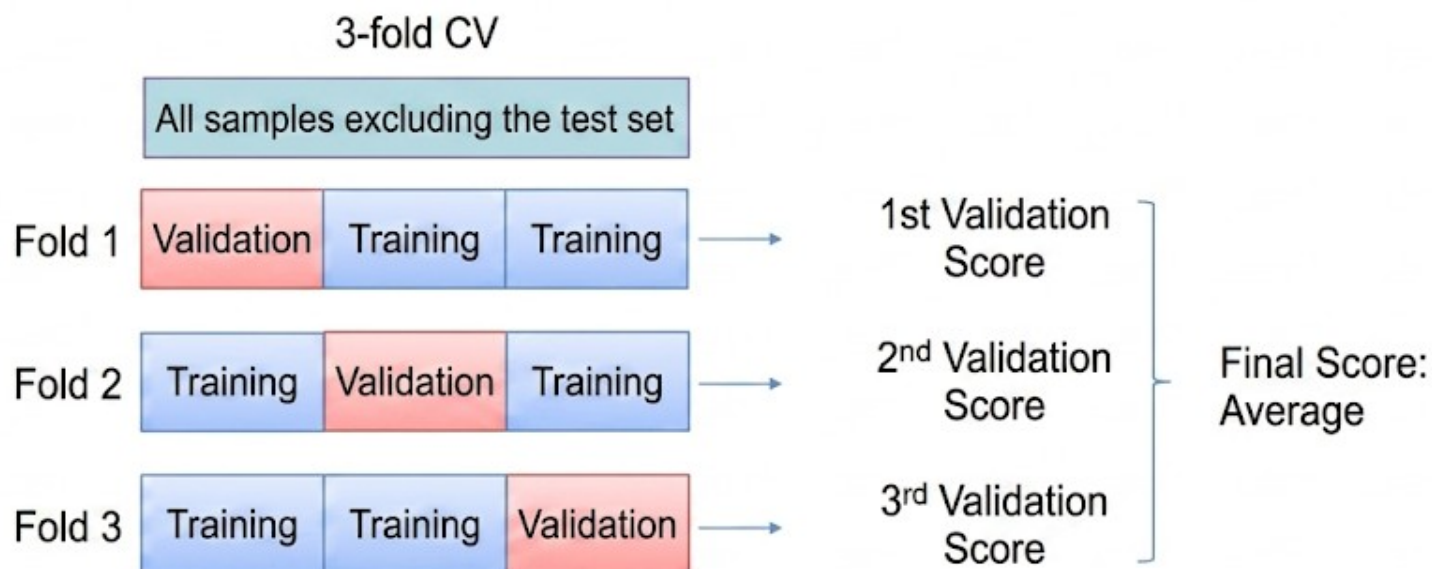
K-fold Cross-Validation

Problem: If the data is scarce, the validation set might contain too few samples, thus failing to represent the data statistically.

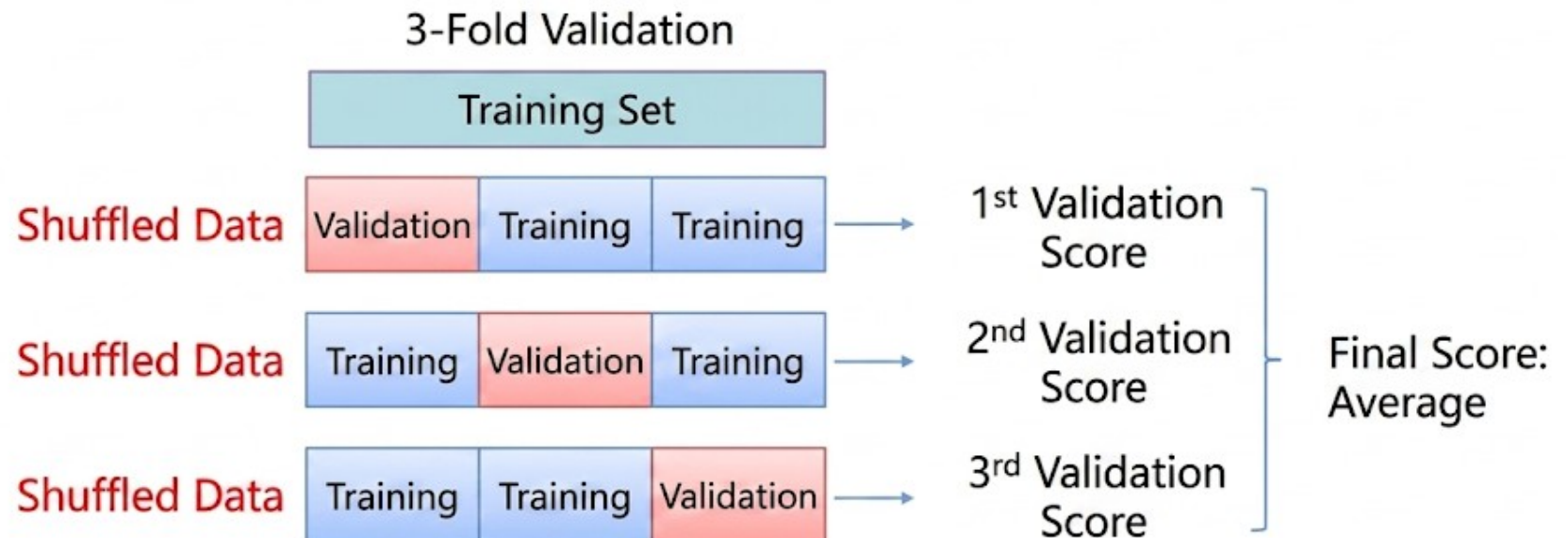
This problem is easily detected: if performing different random shuffles before partitioning the data results in a large difference in final model performance, then this problem exists.

Next, *K*-fold Cross-Validation and Repeated *K*-fold Cross-Validation will be introduced, which are two methods to solve this problem.

K-fold Cross-Validation



Repeated K-Fold Validation with Shuffled Data

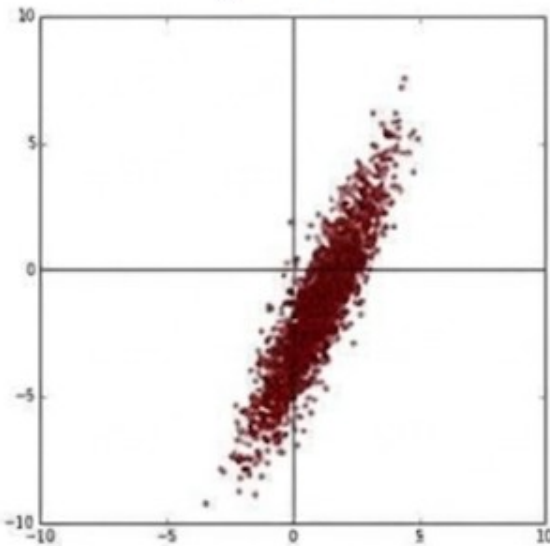


Data Preprocessing

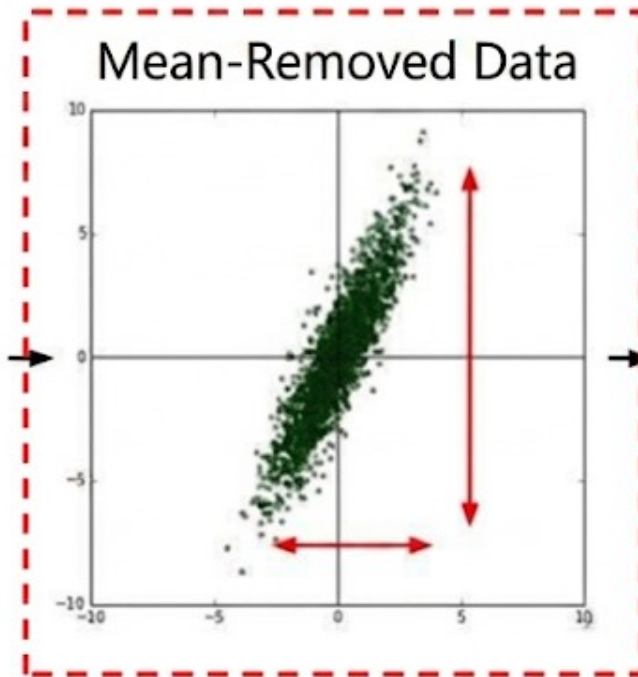
Can the data be used directly?
What are the preprocessing methods?

Data Preprocessing - 1

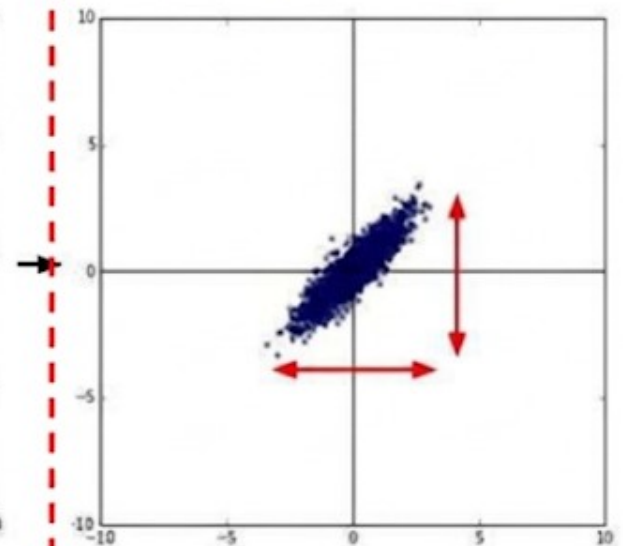
Original Data



Mean-Removed Data

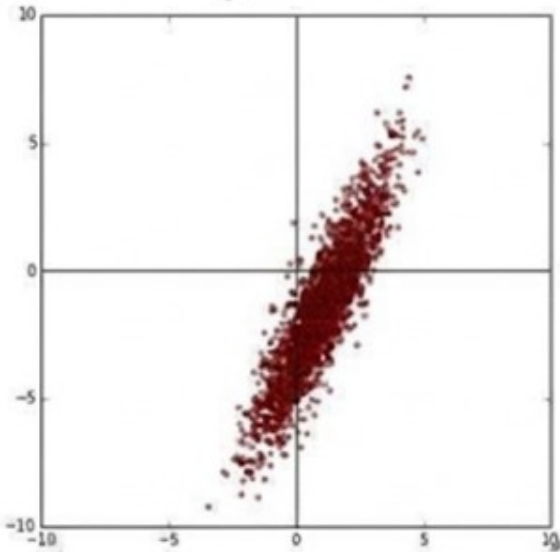


Normalized Data

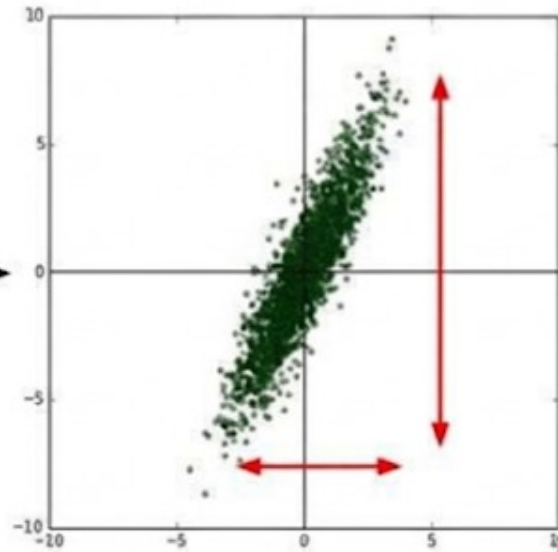


Data Preprocessing - 1

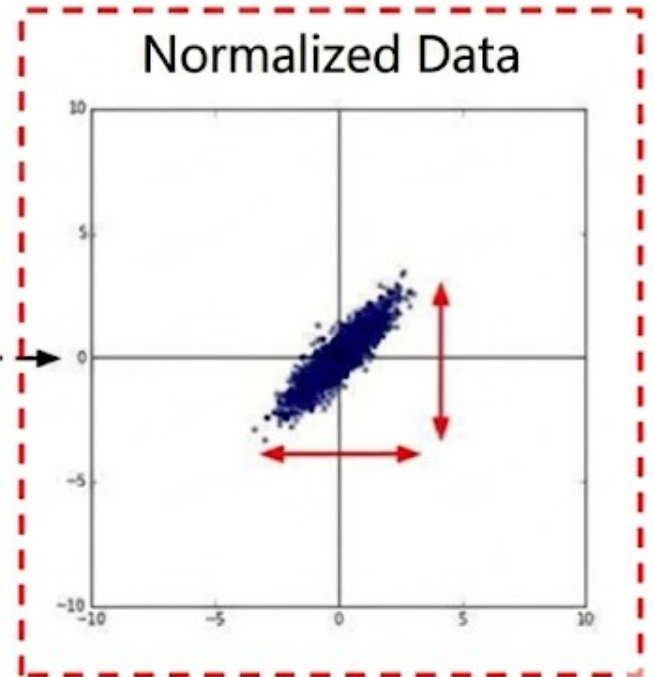
Original Data



Mean-Removed Data

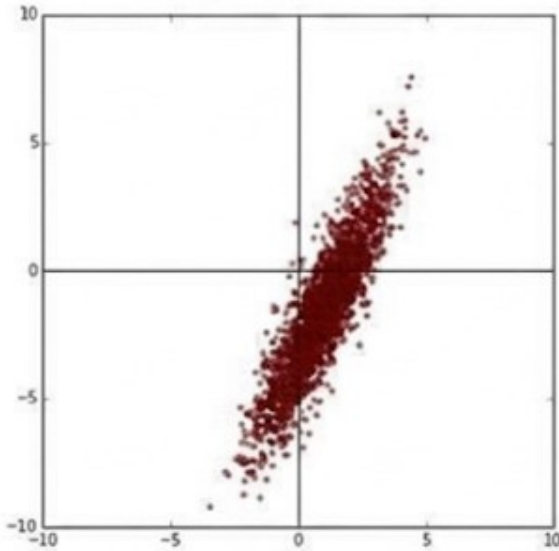


Normalized Data

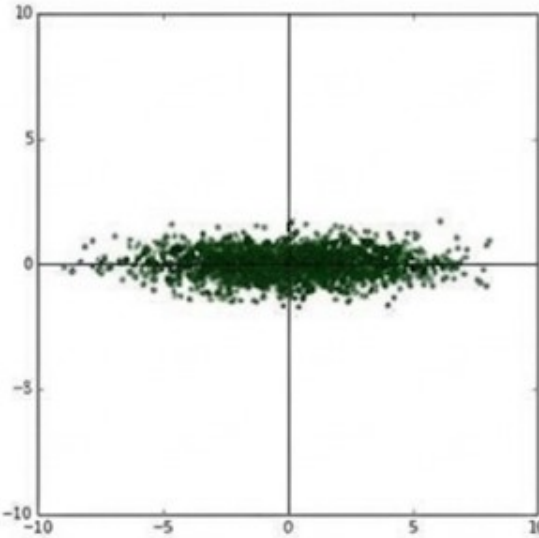


Data Preprocessing - 2

Original Data

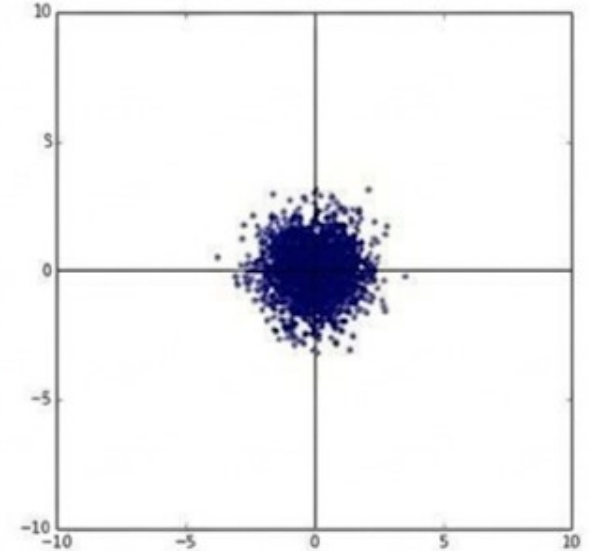


Decorrelated Data



(Data covariance matrix is
a diagonal matrix)

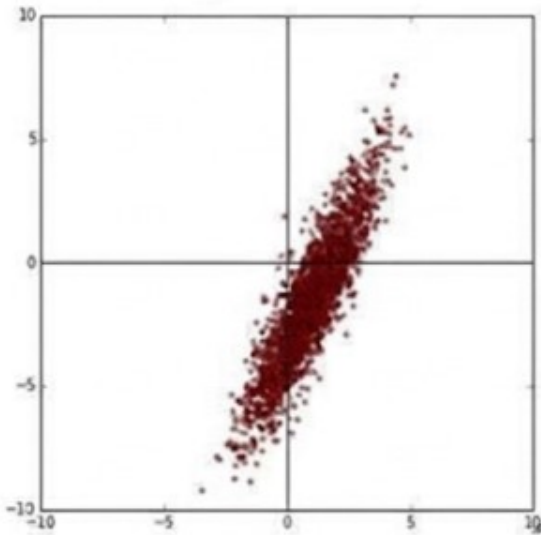
Whitened Data



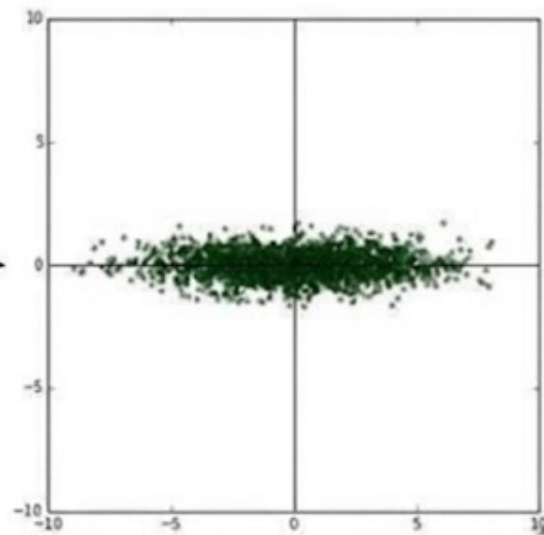
(Covariance matrix is
an identity matrix)

Data Preprocessing - 2

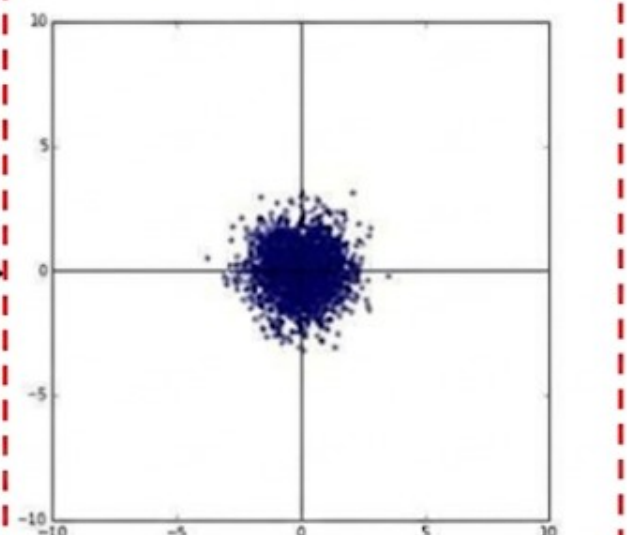
Original Data



Decorrelated Data



Whitened Data



(Covariance matrix is diagonal) (Covariance matrix is identity)

Linear Classifier Experience

<http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>